

Preface

This master's thesis marks the completion of my five-year Master's programme in Cybernetics and Robotics at the Department of Engineering Cybernetics, Norwegian University of Science and Technology (NTNU).

The motivation for this thesis came from my specialization project, where I first worked on biologically plausible alternatives to backpropagation and implemented perturbation-based learning methods. That work showed that perturbation methods could train neural networks on supervised learning tasks, and suggested that several theoretical and practical questions in this area remained open. This became the starting point for the present thesis. Although the objectives and contributions of this thesis are substantially different, both works build on the same underlying motivation: understanding how learning in the brain can inspire alternatives to backpropagation in artificial neural networks. Parts of the background material are therefore based on the specialization project.

I would like to express my sincere gratitude to my supervisor, Bjarne André Grimstad, for his guidance, feedback, and insightful discussions throughout the work on this thesis. This thesis started from a very open research direction, with a large number of possible paths to follow, and Bjarne was especially helpful in guiding me toward the questions that were actually worth pursuing. His support has been highly valuable, both in shaping the direction of the project and in improving the clarity of the final work.

I would also like to thank my family and friends for their support throughout my studies. Finally, I want to thank NTNU, the Department of Engineering Cybernetics, and the city of Trondheim for five memorable years.

Trondheim, June 2026

Mats Wiig Martinussen

Abstract

Even though artificial intelligence has achieved remarkable results over the past decade, several fundamental challenges remain unresolved. Current systems are highly energy intensive, struggle to learn continuously without interfering with existing knowledge, often fail to generalize reliably to novel situations, and lack the grounded causal understanding that comes from interacting with the world. If we want to address these problems, a natural place to look for inspiration is biology, and in particular the human brain, where many of these capabilities already exist. If future AI systems take stronger inspiration from biological architectures, they will also need learning rules that can operate within such architectures. Backpropagation trains today’s neural networks extremely well, but it is difficult to reconcile with biological learning. This motivates the search for biologically plausible learning algorithms.

This thesis studies perturbation-based learning, a branch of biologically plausible learning where random perturbations are added to a network’s weights or neural activity, and the resulting change in performance is used to decide whether these perturbations should be reinforced or reversed. The thesis first reviews the existing literature and identifies an unresolved gap in the relationship between the two main forms of perturbation learning. Building on an idea from Bayesian neural networks, it then addresses this gap theoretically by deriving a theorem showing that one of these learning families can be viewed as a variance-reduced version of the other. During this derivation, a new method called input-scaled node perturbation is introduced. To the best of our knowledge, neither the theoretical result nor the method has been derived before.

The empirical results provide evidence that this theoretical result also matters in practice. As the tested tasks become more complex, the lower-variance family of methods becomes increasingly stronger than the higher-variance alternative, giving empirical support to the theoretical result. The results also suggest that input-scaled node perturbation improves on the existing perturbation methods it is compared against, especially on the more challenging tasks. This indicates that the theoretical contribution is not only analytically meaningful, but also relevant for learning performance. At the same time, backpropagation remains far superior on the most challenging tasks studied here. The thesis therefore does not present perturbation-based learning as a replacement for backpropagation, but as a biologically motivated learning principle with real capability and room for further development, including the kind of theoretically motivated improvement developed in this work.

Sammendrag

Selv om kunstig intelligens har oppnådd bemerkelsesverdige resultater det siste tiåret, står feltet fortsatt overfor flere grunnleggende utfordringer. KI krever store mengder energi, har vanskeligheter med å tilegne seg ny kunnskap uten å overskrive det som allerede er lært, generaliserer ofte dårlig til nye situasjoner som skiller seg fra treningsdataene, og mangler den kausale forståelsen av verden som utvikles gjennom erfaring og samhandling med den fysiske verden. Dersom vi ønsker å løse disse problemene, er det naturlig å hente inspirasjon fra biologien, og særlig fra den menneskelige hjernen, hvor mange av disse egenskapene allerede finnes. Hvis fremtidige KI-systemer i større grad skal bygge på biologisk inspirerte arkitekturer, trenger de også læringsalgoritmer som kan fungere innenfor slike arkitekturer. Backpropagation trener kunstige nevrale nettverk svært effektivt, men er vanskelig å forene med biologisk læring. Dette motiverer søket etter biologisk plausible læringsalgoritmer.

Denne masteroppgaven studerer perturbasjonsbasert læring, en retning innen biologisk plausibel læring der tilfeldig støy legges til et nettverks vektorer eller nevrale aktivitet. Endringen i ytelse som følger av denne støyen, brukes deretter til å avgjøre om perturbasjonene skal forsterkes eller reverseres. Oppgaven gjennomgår først den eksisterende litteraturen innenfor dette feltet, hvor den identifiserer et uavklart spørsmål knyttet til forholdet mellom de to hovedformene for perturbasjonsbasert læring. Med utgangspunkt i en idé fra bayesianske nevrale nettverk adresseres dette spørsmålet deretter gjennom utledningen av et teorem som viser at den ene familien av læringsmetoder kan forstås som en variansreduert versjon av den andre. I denne utledningen introduseres også en ny metode kalt input-skalert nodeperturbasjon. Så vidt vi kjenner til, har verken det teoretiske resultatet eller metoden blitt utledet tidligere.

De empiriske resultatene gir støtte for at teoremet også har praktisk betydning. Etter hvert som oppgavene blir mer komplekse, viser metodene med lavere varians tydelig bedre læringsytelse enn den andre formen. Resultatene tyder også på at input-skalert nodeperturbasjon fungerer bedre enn de eksisterende perturbasjonsmetodene den sammenlignes med, særlig på de mer utfordrende oppgavene. Dette tyder på at det teoretiske bidraget også fører til en metode med forbedret læringsytelse. Med det sagt fungerer backpropagation fortsatt langt bedre enn samtlige perturbasjonsbaserte algoritmer på de mest utfordrende oppgavene. Denne masteroppgaven fremstiller derfor ikke perturbasjonsbasert læring som en erstatning for backpropagation, men som en biologisk motivert læringsalgoritme som er kapabel til å lære ikke-trivielle oppgaver, samtidig som den har tydelig rom for videre utvikling, inkludert den typen teoretisk motivert forbedring som utvikles i dette arbeidet.

Table of Contents

Preface	i
Abstract	ii
Sammendrag	iii
List of Figures	vi
List of Tables	vi
Acronyms	viii
1 Introduction	1
1.1 Research Questions and Objectives	3
1.2 Structure of the Thesis	3
2 Theory	4
2.1 How the brain learns	4
2.1.1 Hebbian learning	5
2.1.2 Neuromodulators and three-factor learning	5
2.1.3 Stochasticity in biological learning	6
2.2 Learning in artificial neural networks	6
2.2.1 Feedforward neural networks and supervised learning	6
2.2.2 Backpropagation	7
2.2.3 Why backpropagation is biologically implausible	9
2.3 Biologically plausible alternatives to backpropagation	9
2.4 Weight perturbation	10
2.5 Node perturbation	13
2.6 Other variants of Weight Perturbation and Node Perturbation	14
2.7 Existing comparisons of weight and node perturbation	17
2.8 Literature gap	18
2.9 The local reparameterization trick	18
2.10 Summary of the Theory	21
3 Relating Weight and Node Perturbation Through Induced Noise	22
3.1 Pre-activation noise induced by WP	22
3.2 Input-scaled node perturbation	23

3.3	Variance reduction theorem	25
3.4	Why IS-NP may improve over existing NP variants	27
3.5	Summary of theoretical contribution	28
4	Method	29
4.1	Learning problems and datasets	30
4.1.1	Synthetic sinus regression	30
4.1.2	California housing regression	30
4.1.3	MNIST classification	31
4.1.4	CIFAR-10 classification	31
4.1.5	Scaled-input diagnostics	31
4.2	Model architecture and initialization	32
4.3	Algorithms	32
4.4	Evaluation metrics	36
4.4.1	Training and test performance	36
4.4.2	Directional alignment	37
4.4.3	Estimator variance	38
4.5	Hyperparameter selection and experimental protocol	39
4.6	Declaration of AI	42
5	Results	42
5.1	Sinus Regression	42
5.2	California Housing Regression	44
5.3	MNIST Classification	45
5.4	CIFAR-10 Classification	46
5.5	Scaled Input Sensitivity	48
6	Discussion	48
6.1	Overview of the main findings	48
6.2	Practical capability and limits of perturbation-based learning	49
6.3	Weight versus node perturbation	49
6.4	IS-NP versus other node perturbation methods	50
6.5	Limitations	51
6.6	Future work	51
7	Conclusion	52

8 Sustainability	53
-------------------------	-----------

Appendix	54
-----------------	-----------

A	Checkpoint-Specific Estimator Diagnostics	54
A.1	Sinus Regression	54
A.2	Scaled Input Sensitivity	56
A.3	California Housing Regression	58
A.4	MNIST Classification	60
A.5	CIFAR-10 Classification	62
B	Full Result Summary Tables	64
C	Expected Equivalence Between WP and IS-NP	69

List of Figures

1	Results for the sinus regression task.	43
2	Results for the California housing regression task.	44
3	Results for the MNIST classification task.	46
4	Results for the CIFAR-10 classification task.	47
5	Effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$	48
6	Checkpoint-specific cosine similarity for the sinus regression task.	55
7	Checkpoint-specific estimator variance for the sinus regression task.	55
8	Checkpoint-specific cosine similarity for the scaled-input sinus diagnostic.	57
9	Checkpoint-specific estimator variance for the scaled-input sinus diagnostic.	57
10	Checkpoint-specific cosine similarity for the California housing regression task.	59
11	Checkpoint-specific estimator variance for the California housing regression task.	59
12	Checkpoint-specific cosine similarity for the MNIST classification task.	61
13	Checkpoint-specific estimator variance for the MNIST classification task.	61
14	Checkpoint-specific cosine similarity for the CIFAR-10 classification task.	63
15	Checkpoint-specific estimator variance for the CIFAR-10 classification task.	63

List of Tables

1	Comparison of perturbation-based update rules in the literature.	16
2	Datasets used in the empirical evaluation.	32
3	Model architectures used in the empirical evaluation.	32
4	Training and estimator-diagnostic settings for each learning problem.	36

5	Local hyperparameter grids used during tuning. For perturbation methods, the grid is the Cartesian product of the listed learning rates and perturbation scales. The selected setting is the final configuration used for the thesis results.	41
6	Main summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.	42
7	Main summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.	45
8	Main summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.	45
9	Main summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.	47
10	Full summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.	65
11	Full summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.	66
12	Full summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.	67
13	Full summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.	68

Acronyms

AI	Artificial intelligence
BP	Backpropagation
CIFAR-10	Canadian Institute for Advanced Research 10-class image dataset
CL1	Cortical Labs CL1 biological computing system
CNN	Convolutional neural network
IS-NP	Input-scaled node perturbation
KL	Kullback–Leibler
MLP	Multilayer perceptron
MNIST	Modified National Institute of Standards and Technology handwritten digit dataset
MSE	Mean squared error
NP	Node perturbation
NTNU	Norwegian University of Science and Technology
REINFORCE	Reward Increment equals Nonnegative Factor times Offset Reinforcement times Characteristic Eligibility
ReLU	Rectified linear unit
SGD	Stochastic gradient descent
SGVB	Stochastic gradient variational Bayes
WP	Weight perturbation

1 Introduction

Artificial intelligence has undergone a dramatic transformation over the past decade, emerging as one of the most powerful technological forces of our time. Recent advances in deep learning and large-scale optimization have produced systems capable of performing tasks once considered exclusive to human intelligence. Transformer-based language models can generate coherent text, write code, and assist in complex reasoning tasks (Vaswani et al., 2017; Brown et al., 2020). Reinforcement learning systems have achieved superhuman performance in games such as Go (Silver et al., 2016), while breakthroughs in scientific machine learning have enabled major progress on long-standing challenges such as protein structure prediction (Jumper et al., 2021). Taken together, these developments suggest that artificial intelligence is not only progressing rapidly, but may be approaching a level of capability that raises fundamental questions about the feasibility of artificial general intelligence.

Despite this progress, it is highly debated whether anything resembling artificial general intelligence can be reached simply by scaling today’s methods. Modern AI systems are increasingly powerful, but they still differ from biological intelligence in several important ways. In particular, they continue to face challenges that suggest that more data, larger models, and more compute may not be the full answer. Among the most important are:

- **Energy intensity and consumption.** Modern AI systems require enormous amounts of computation and energy. Until around 2012, compute demand for major AI models roughly followed Moore’s law, doubling approximately every 24 months. Since then, it has accelerated sharply, with recent frontier models doubling compute demand as quickly as every 2 months (Mehonic and Kenyon, 2022). This energy problem becomes especially clear at biological scale: a digital simulation of an artificial neural network approximately the size of the human brain would consume 7.9MW. This illustrates how energy-intensive conventional digital approaches can become, and suggests that simply scaling current systems may face serious efficiency limits.
- **Lack of continual learning.** An intelligent system should be able to learn from new experiences and information over time. For example, an AI assistant should be able to adapt to a user’s preferences, and a model deployed in the real world should be able to incorporate important new events after its initial training. In current AI systems, however, updating a model after deployment is often difficult because new learning can interfere with old knowledge, a problem known as catastrophic forgetting (McCloskey and Cohen, 1989). As a result, many current systems rely on a separate training phase followed by a deployment phase, where the model’s knowledge is largely fixed.
- **Limited generalization and abstraction.** Current AI systems often perform well on tasks that resemble their training data, but become less reliable in novel situations. One reason is that models may rely on statistical regularities or superficial correlations rather than learning abstract concepts that transfer. For example, a model trained to recognize chairs may rely on common features such as four legs, a backrest, or a familiar shape. If it is then shown an unusual chair, such as an artistic design with one thick central support or a shape unlike the examples seen during training, it may fail to recognize it as a chair, even though its function as something to sit on is clear. Robust intelligence requires concepts that generalize beyond the most common patterns in the training data (Dehaene, 2021).
- **Lack of grounded causal understanding.** A final, more fundamental challenge is that many current AI systems learn primarily from static data rather than through interaction with the world. Passive data can be extremely powerful, as language models clearly demonstrate, but it does not provide the same kind of grounded experience as learning through action and consequence. For example, a language model may learn the word “heavy” from thousands of textual contexts, but this is different from learning heaviness by lifting an object and experiencing how weight affects movement. Such systems may therefore learn rich statistical relations between concepts without grounding those concepts in direct interaction with the environment. The concern is that this may limit the kind of intelligence they can

develop: impressive pattern recognition from passive data is not necessarily the same as causal understanding built through experience (Bisk et al., 2020).

Biological systems provide a proof of existence that these challenges can be handled in practice. The energy contrast is especially striking: while a digital simulation of an artificial neural network approximately the size of the human brain would consume 7.9 MW, the human brain operates at roughly 20 W, corresponding to a difference of about six orders of magnitude (Mehonic and Kenyon, 2022). Biological systems also learn continuously. Humans do not have a separate training phase followed by a fixed deployment phase; we learn throughout life, incorporate new experiences, and usually do so without overwriting existing knowledge in the catastrophic way observed in artificial neural networks. Similarly, humans often form abstract concepts rather than only memorizing superficial regularities. A person can recognize an unusual chair as a chair, even if it differs from most chairs seen before, because the concept is tied to its function as something to sit on rather than only to features such as four legs or a familiar shape (Dehaene, 2021). Finally, biological understanding is grounded in interaction with the world. We do not learn concepts such as heaviness only from descriptions, but by acting, sensing consequences, and building causal expectations from experience. These properties make biological intelligence a natural reference point when searching for more efficient, adaptive, and general learning systems.

Since biological systems handle many of the challenges above, it is natural to ask whether future AI systems should take more direct inspiration from biological neural architectures. The point is not that the brain can be copied directly today, but that biology may contain computational principles that current digital systems do not exploit well. This has motivated work on spiking neural networks, recurrent dynamics, neuromorphic hardware, and biological substrates. A recent example is Cortical Labs’ CL1 system, which integrates living neurons with silicon hardware and allows biological neural networks to interact with software in real time. Building on earlier work where cultured neurons learned to play Pong (Kagan et al., 2022), Cortical Labs has also demonstrated CL1 interacting with the game *Doom* (Cortical Labs, 2026). Such systems are still at a very early stage, but they illustrate that biological computation is no longer only a theoretical idea.

However, more biologically inspired architectures also require learning rules that are compatible with them. The dominant learning algorithm in modern AI is backpropagation, which trains virtually all current neural networks by computing exact gradients through a backward pass. This is extremely effective in artificial neural networks, but the mechanisms it relies on break sharply with biological learning: neurons and synapses do not appear to have access to precise neuron-specific error derivatives, a separate recursive backward computation, or feedback pathways that use exact copies of downstream weights in reverse (Crick, 1989; Lillicrap et al., 2020). This creates a central problem. If more biologically inspired architectures are ever to become useful learning systems, they cannot only borrow biological structure; they also need learning rules that can plausibly operate within that structure. In such settings, biologically plausible learning algorithms, which are the focus of this thesis, are not just an interesting alternative to backpropagation, but a necessary part of the broader search for more brain-like artificial intelligence. To provide a controlled setting for analysing such methods, this thesis tests them in standard artificial neural networks and compares them against backpropagation. However, because the methods are biologically plausible, they are also candidates for implementation in more biologically inspired architectures.

Several biologically plausible alternatives to backpropagation have been proposed, including feedback alignment, predictive coding, and the forward-forward algorithm (Lillicrap et al., 2016; Nøkland, 2016; Rao and Ballard, 1999; Whittington and Bogacz, 2017; Hinton, 2022). This thesis focuses on perturbation-based learning, a conceptually simple family of methods where random noise is injected into the weights or neural activity and then correlated with a scalar performance signal. If the perturbation improves performance, it is reinforced; if it worsens performance, it is suppressed. This idea has been studied as a way to train neural networks since at least the early 1990s, and several variants have been proposed since then (Widrow and Lehr, 1990; Flower and Jabri, 1992; Williams, 1992; Werfel et al., 2003). These methods differ in where the noise is injected, how the performance signal is formed, and how the resulting update is assigned to weights.

Although perturbation-based learning is therefore an established research direction, several central

questions remain unresolved. Recent work is largely organized around two main families: weight perturbation, where noise is injected into the parameters, and node perturbation, where noise is injected into neural activity. However, the relationship between these two families is still not fully understood. In addition, the practical capability of perturbation methods remains unclear: existing papers show that they can train neural networks, but do not provide a systematic picture of what types of tasks they can solve well, where they become inefficient, and where they begin to break down. Finally, despite the long history of perturbation-based learning, the literature is still relatively small compared with other alternatives to backpropagation. This leaves room both for a clearer theoretical comparison of existing methods and for the development of new perturbation-based learning rules.

These gaps make perturbation-based learning a natural focus for this thesis. The field is established enough that there is a meaningful literature to build on, but open enough that important theoretical and empirical questions remain unresolved. This motivates the three research questions and objectives stated below.

1.1 Research Questions and Objectives

The objective of this thesis is to investigate perturbation-based learning methods as biologically plausible alternatives to backpropagation. The thesis combines literature analysis, theoretical development, and empirical evaluation.

- **What has been proposed and theoretically established in the literature on perturbation-based learning?** This question is addressed by reviewing and comparing existing perturbation-based learning methods, with emphasis on how different approaches inject noise, form learning signals, and update weights. The objective is to establish what has already been proposed and theoretically understood, identify the methods that appear most central in the literature, and clarify where the existing understanding remains incomplete.
- **How can perturbation-based learning be further developed?** This question is addressed by first clarifying an incomplete part of the existing work identified in the literature study. This theoretical analysis is then used to motivate a theoretically grounded variant of node perturbation. The objective is to further develop perturbation-based learning by strengthening its theoretical basis, while also identifying a modification that may improve estimator quality and learning performance.
- **What is the practical capability of perturbation-based learning?** This question is addressed by evaluating the perturbation-based methods identified as most central in the literature study, together with the new method derived in this thesis, and comparing them against backpropagation across supervised learning tasks of increasing complexity. The objective is to understand both when these methods work well and when they begin to break down, using backpropagation as the standard reference method, while also comparing the perturbation methods against each other.

The three questions are closely connected. The literature gap identified through the first question provides the basis for the theoretical development in the second, while the empirical evaluation in the third question is used to assess the practical relevance of both existing perturbation methods and the new development introduced in this thesis.

1.2 Structure of the Thesis

This thesis is structured as follows. Chapter 2 first presents relevant background on biological learning, before deriving backpropagation and explaining why it is biologically implausible. The chapter then turns to perturbation-based learning, reviews the current literature, identifies the main literature gap, and finally introduces relevant ideas from Bayesian neural networks as inspiration for addressing this gap. Chapter 3 relates weight and node perturbation through induced

noise. In this chapter, we derive a result that clarifies the gap identified in Chapter 2, and use this analysis to introduce a new perturbation-based learning method. Chapter 4 describes the empirical setup, including the learning problems, algorithms, hyperparameters, and evaluation metrics used in the experiments. Chapter 5 presents the empirical results, while Chapter 6 discusses these results in relation to the theory, the proposed method, and the research objectives. Finally, Chapter 7 concludes the thesis by answering the research questions.

2 Theory

Chapter 1 motivated why biologically plausible learning algorithms are interesting. To understand why perturbation methods are relevant in this context, we need to establish two things: first, that the current dominant training algorithm, backpropagation, is difficult to reconcile with biological constraints; and second, that perturbation-based methods satisfy these constraints more naturally. This chapter therefore begins by introducing the biological learning principles relevant for the thesis, drawing on work on synaptic plasticity, neuromodulation, and three-factor learning rules (Hebb, 1949; Schultz et al., 1997; Frémaux and Gerstner, 2016; Gerstner et al., 2018). It then reviews learning in artificial neural networks, derives the backpropagation update (Rumelhart et al., 1986), and explains why backpropagation is considered biologically implausible (Crick, 1989; Lillicrap et al., 2020). With this background in place, the chapter turns to perturbation-based learning (Williams, 1992; Werfel et al., 2003; Hiratani et al., 2022; Züge et al., 2023) and addresses the first research objective by reviewing what has been proposed and theoretically established in this literature, and identifying the remaining gaps that motivate the theoretical contribution in Chapter 3.

Parts of the background material in this chapter build on my earlier project thesis on biologically plausible alternatives to backpropagation (Martinussen, 2025). In particular, Sections 2.1 and 2.2, together with the introductory derivations of weight perturbation and node perturbation in Sections 2.4 and 2.5, reuse and adapt theory that remains unchanged between the two works. The objectives, theoretical contribution, and empirical evaluation of the present thesis are substantially different, but this shared background provides the necessary foundation for both.

2.1 How the brain learns

Learning can be understood as the process by which the brain builds and updates internal models of the world. Dehaene (2021) describes learning as constructing such models from experience. These models may represent, for example, the layout of a neighbourhood, the way our limbs respond when we move, or the rules of a language we use in daily life. They are not fixed at birth, but gradually formed through interaction with the environment. In this sense, learning means changing the brain’s internal representations so that they better reflect the structure of the world.

These internal representations are implemented in networks of neurons connected by synapses. Each synapse defines a directed connection from a presynaptic neuron, which sends the signal, to a postsynaptic neuron, which receives it. Neurons transmit information using short electrical impulses, and their likelihood of firing is determined by the signals they receive from other neurons. Synapses determine how strongly one neuron influences another. A strong synapse makes activity in the presynaptic neuron more likely to affect the postsynaptic neuron, while a weak synapse has little influence. When large numbers of neurons interact, their joint activity can encode structured information, such as the sound of a familiar word or the spatial structure of a known street. These learned representations are therefore not stored as explicit symbolic rules, but as distributed patterns of neural activity shaped by synaptic connections.

The question is then how these internal models change. Dehaene also describes learning as “adjusting the parameters of a mental model” (Dehaene, 2021). In the brain, the natural analogue of such parameters is the strength of synaptic connections. Synapses are plastic: their strengths can increase or decrease depending on experience (Kandel et al., 2021). When synaptic strengths change, the influence neurons have on each other also changes, and this modifies the activity pat-

terns the network can produce. Learning in the brain can therefore be viewed as the adjustment of synaptic connections in a large neural network, so that the representations encoded by the network become better adapted to the environment.

There are many biological mechanisms that can modify synaptic strengths, and neuroscience contains several theories of how such mechanisms give rise to learning. The purpose of this section is not to give a complete account of biological plasticity, but to introduce the principles that are most relevant for the learning algorithms studied in this thesis. No single theory fully explains how the brain learns, but the ideas discussed below represent influential mechanisms that have motivated biologically grounded learning algorithms, including perturbation-based learning.

2.1.1 Hebbian learning

Hebbian learning is a central idea in theories of synaptic plasticity. The basic principle, introduced by Hebb (1949), is that a synapse should be strengthened when activity in the presynaptic neuron consistently helps activate the postsynaptic neuron. Abstractly, this can be written as

$$\Delta w_{ij} = \eta f(x_i) f(x_j),$$

where w_{ij} denotes the connection strength from neuron i to neuron j , x_i and x_j denote their activities, $f(\cdot)$ allows the update to depend nonlinearly on neural activity, and $\eta > 0$ sets the learning rate. The important point is that this is a local rule: the synapse does not need access to global information about the task or the network, but only to signals associated with the two neurons it connects.

Although simple, this type of local learning can produce meaningful structure. Since Hebbian updates strengthen connections between neurons that are active together, they reinforce statistical regularities in the input. Oja’s rule provides a classic example of this idea in a stabilized form: by adding a normalization-like correction to the basic Hebbian mechanism, it allows a single neuron to learn the first principal component of the input distribution (Oja, 1982). Hebbian learning therefore shows that purely local, unsupervised plasticity can support nontrivial forms of representation learning.

However, the limitation of this learning rule is that it contains no feedback about the outcome of the network state. The synapse changes when pre- and postsynaptic activity are correlated, but the update does not depend on whether the resulting action, prediction, or behaviour was good or bad. It therefore cannot by itself explain how the brain learns from reward, error, success, or failure. For this reason, Hebbian plasticity is an important foundation for synaptic learning, but not a complete account of learning in the brain.

2.1.2 Neuromodulators and three-factor learning

Local synaptic activity alone cannot explain how the brain learns from success, failure, reward, or error. For outcome-dependent learning, synapses also need some form of feedback about whether recent neural activity or behaviour was useful. One important class of such feedback signals is provided by neuromodulators. Unlike ordinary point-to-point synaptic communication, neuromodulatory signals can be broadcast to larger brain regions, allowing many neurons and synapses to receive the same information. Dopamine is a central example. Activity in midbrain dopamine neurons has been associated with reward-prediction errors: it tends to increase when outcomes are better than expected and decrease when outcomes are worse than expected (Schultz et al., 1997). Dopamine can therefore act as a global scalar feedback signal, carrying information about whether recent neural activity or behaviour led to a better or worse outcome than expected.

A common theory of outcome-dependent synaptic plasticity is that this global feedback signal is combined with local Hebbian-like information at each synapse. This gives rise to three-factor learning rules, where synaptic change depends on a local presynaptic term, a local postsynaptic term, and a global modulatory signal (Frémaux and Gerstner, 2016; Gerstner et al., 2018):

$$\Delta w_{ij} = \eta f_i(x_i) f_j(x_j) M. \tag{1}$$

Here, w_{ij} is the synaptic weight from neuron i to neuron j , $f_i(x_i)$ and $f_j(x_j)$ are potentially nonlinear functions of the pre- and postsynaptic states, M is a global modulatory signal, and η is a learning-rate parameter. The local term $f_i(x_i)f_j(x_j)$ can be interpreted as an eligibility trace: a synapse-specific signal determined by recent pre- and postsynaptic activity. Intuitively, the global signal M says whether the outcome was good or bad, while the local eligibility trace indicates how relevant that synapse was for producing the activity that led to the outcome.

Three-factor learning rules address the main limitation of pure Hebbian plasticity by allowing synaptic change to depend on outcome feedback as well as local neural activity. A synapse is therefore not modified only because pre- and postsynaptic neurons were active together, but also according to whether the resulting activity or behaviour was associated with a better or worse outcome. In this way, three-factor rules provide a biologically motivated account of how global feedback can be assigned to individual synapses, making them an influential model of outcome-dependent learning in the brain (Frémaux and Gerstner, 2016; Gerstner et al., 2018).

2.1.3 Stochasticity in biological learning

Biological learning takes place in a noisy environment. Neural firing is variable, synaptic transmission is stochastic, and the same stimulus does not always produce exactly the same neural response. Rather than being only a nuisance, this stochasticity provides a source of exploration. When random fluctuations in neural activity or synaptic efficacy are correlated with later reward or error signals, the nervous system can reinforce fluctuations that improve performance and suppress those that do not (Seung, 2003). This idea directly motivates perturbation-based learning algorithms, where random perturbations are introduced into weights or neural activities and then correlated with a global scalar learning signal.

The biological principles relevant for this thesis can therefore be summarized as follows. The learning rules considered here use information from the presynaptic neuron, the postsynaptic neuron, and a global scalar feedback signal, such as a reward, error, or neuromodulatory signal. In addition, stochastic fluctuations in weights or neural activity can provide the exploration needed for learning. Together, pre- and postsynaptic information, global feedback, and stochastic perturbations form the biological motivation for the perturbation-based learning rules studied later in this chapter.

2.2 Learning in artificial neural networks

2.2.1 Feedforward neural networks and supervised learning

An artificial neural network consists of layers of artificial neurons connected by weights. In a feedforward network, information moves in one direction: from the input layer, through one or more hidden layers, and finally to the output layer. Each neuron in a layer receives signals from neurons in the previous layer, and the weights determine how strongly each previous neuron influences it. In this sense, weights play the same abstract role as synapses in biological neural networks. For each neuron, the incoming activations are combined using these weights and a bias term, before the result is passed through a nonlinear activation function. Following the standard notation for feedforward networks (Goodfellow et al., 2016), this can be written as

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell, \quad x_j^\ell = \phi(z_j^\ell). \quad (2)$$

Here, z_j^ℓ is the pre-activation of neuron j in layer ℓ , $x_i^{\ell-1}$ is the activity of neuron i in the previous layer, W_{ij}^ℓ is the corresponding weight, b_j^ℓ is a bias, and $\phi(\cdot)$ is the activation function. Repeating this operation over many layers gives a flexible parameterized function from inputs to outputs. Although this structure is inspired by the brain, where neurons are connected through synapses, it is only a highly simplified abstraction of biological neural networks. In the brain, information does not simply flow in one direction through a fixed sequence of layers, but is shaped by recurrent connections, feedback loops, temporal dynamics, and many other biological mechanisms. Despite

the simplicity of this construction, such networks are highly expressive. The universal approximation theorem shows that feedforward networks can approximate any continuous function on a compact domain, provided that the network is sufficiently large (Hornik, 1991). The setting considered here is supervised learning. We are given examples $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is an input and $\mathbf{y}$ is the desired output. A network with parameters $\boldsymbol{\theta}$, collecting all weights and biases, produces a prediction $\hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x})$. The quality of this prediction is measured by a loss function $L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x}))$. The aim is not simply to reduce the loss on the observed training examples, but to learn parameters that also perform well on unseen examples drawn from the same underlying distribution. This generalization objective can be written as

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \mathbb{E}_{(\mathbf{x}, \mathbf{y})} [L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x}))]. \quad (3)$$

In principle, learning would amount to solving this optimization problem and directly finding $\boldsymbol{\theta}^*$. For modern neural networks, however, this is not tractable. The parameter space is large, the loss landscape is generally non-convex, and the full data-generating distribution is unknown. Instead, learning is performed through iterative updates. Starting from an initial parameter vector, a learning algorithm repeatedly chooses an update direction and takes a small step:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}.$$

The practical problem is therefore to determine how the update $\Delta \boldsymbol{\theta}$ should be computed. This gives a direct connection to Dehaene’s description of learning as “adjusting the parameters of a mental model” (Dehaene, 2021). In the brain, the relevant parameters are synaptic strengths. In artificial neural networks, they are the weights and biases in $\boldsymbol{\theta}$. Both cases can therefore be viewed as learning by changing the parameters of a network so that its behaviour better matches the structure of the task or environment.

At this level, biological and artificial learning look similar: both involve modifying connection strengths in a network. The difference appears when we look at how these modifications are computed. In modern artificial neural networks, this is usually done using backpropagation (Rumelhart et al., 1986). We therefore next derive the backpropagation update and show why it breaks with the biological learning principles introduced above.

2.2.2 Backpropagation

Backpropagation (Rumelhart et al., 1986) is the standard algorithm for training neural networks. Its purpose is to minimize the objective in Eq. (3) by updating the parameters in the direction of decreasing loss. For a single input–target pair, this requires the gradient

$$\nabla_{\boldsymbol{\theta}} L(\mathbf{y}, \hat{\mathbf{y}}_{\boldsymbol{\theta}}(\mathbf{x})), \quad (4)$$

which tells us how each parameter should be changed in order to reduce the loss. Backpropagation computes this gradient exactly by applying the chain rule backward through the network. Starting from the output layer, error information is propagated layer by layer toward the input, so that each weight receives a gradient describing how it contributed to the final loss.

The derivation begins at the final layer because the loss depends directly on the network output. Any effect that earlier layers have on the loss must pass through the later layers. Therefore, once we know how the loss changes with respect to the output activations, we can obtain the gradients for earlier layers by repeatedly applying the chain rule backward through the network.

The first quantity to compute is the derivative of the loss with respect to each output activation:

$$\frac{\partial L}{\partial x_j^\ell}.$$

Using the activation relation from Eq. (2), this can be converted into a derivative with respect to the corresponding pre-activation. We define this quantity as the error term

$$\delta_j^\ell := \frac{\partial L}{\partial z_j^\ell} = \frac{\partial L}{\partial x_j^\ell} \phi'(z_j^\ell). \quad (5)$$

The term δ_j^ℓ measures how sensitive the loss is to a small change in the pre-activation z_j^ℓ . This is the starting point for the backward propagation of error signals.

From the feedforward definition,

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell,$$

the derivative of z_j^ℓ with respect to an incoming weight is

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}.$$

Applying the chain rule then gives the gradient of the loss with respect to this weight:

$$\frac{\partial L}{\partial W_{ij}^\ell} = \frac{\partial L}{\partial z_j^\ell} \frac{\partial z_j^\ell}{\partial W_{ij}^\ell} \quad (6)$$

$$= \delta_j^\ell x_i^{\ell-1}. \quad (7)$$

Similarly, since the pre-activation depends linearly on the bias,

$$\frac{\partial z_j^\ell}{\partial b_j^\ell} = 1, \quad \frac{\partial L}{\partial b_j^\ell} = \delta_j^\ell.$$

These expressions are not special to the output layer. For any layer ℓ , once the error term δ_j^ℓ is known, the gradients of the incoming weight and bias are given by

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \delta_j^\ell, \quad \frac{\partial L}{\partial b_j^\ell} = \delta_j^\ell.$$

Thus, the forward activations $x_i^{\ell-1}$ and the backward error terms δ_j^ℓ are sufficient to compute the parameter gradients in that layer.

The remaining question is how to compute δ_j^ℓ for hidden layers. In the output layer, the loss depends directly on the activations, so the error term can be computed from the loss. In a hidden layer, however, the loss does not depend directly on x_j^ℓ . A hidden activation affects the loss only through its influence on the next layer. Applying the chain rule gives

$$\frac{\partial L}{\partial x_j^\ell} = \sum_k \frac{\partial L}{\partial z_k^{\ell+1}} \frac{\partial z_k^{\ell+1}}{\partial x_j^\ell} = \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1},$$

where the sum runs over all neurons k in the next layer. Multiplying by the derivative of the activation function then gives the recursive backpropagation equation

$$\delta_j^\ell = \frac{\partial L}{\partial z_j^\ell} = \left(\sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1} \right) \phi'(z_j^\ell).$$

This recursion explains why the algorithm proceeds backward. To compute the error term in layer ℓ , the error terms in layer $\ell + 1$ must already be known. Backpropagation therefore starts at the output layer and recursively propagates error terms backward through the network. Once this backward pass has been completed, the exact gradient $\partial L / \partial \theta$ is available for all parameters.

Substituting the recursive expression for δ_j^ℓ into the weight-gradient formula makes the full structure of a hidden-layer update explicit:

$$\frac{\partial L}{\partial W_{ij}^\ell} = x_i^{\ell-1} \phi'(z_j^\ell) \sum_k W_{jk}^{\ell+1} \delta_k^{\ell+1}. \quad (8)$$

This expression shows that the gradient for a hidden-layer weight depends on three factors: the presynaptic activation $x_i^{\ell-1}$, the local activation derivative $\phi'(z_j^\ell)$, and a weighted sum of error terms from the next layer.

Backpropagation is powerful because it computes the exact gradient of the loss with respect to every parameter, using one forward pass and one backward pass. However, as suggested by the biological learning principles discussed above, the mechanism used by backpropagation differs substantially from how learning is believed to occur in the brain. This is the issue we turn to next.

2.2.3 Why backpropagation is biologically implausible

The biological difficulty becomes clear from the hidden-layer gradient in Eq. (8). The first factor, $x_i^{\ell-1}$, is presynaptic activity, and the activation derivative $\phi'(z_j^\ell)$ is locally available at the post-synaptic neuron. These terms are therefore at least compatible with local synaptic learning. The final factor, however, is different. It requires a precise, signed, neuron-specific error signal from the next layer, weighted by the downstream connections $W_{jk}^{\ell+1}$. This is not information available at the synapse, nor is it a global scalar feedback signal of the kind discussed in three-factor learning. A literal implementation of backpropagation would therefore require biological circuits to transmit exact error derivatives backward through the network. There is no clear evidence that the brain implements such a mechanism. This requirement also leads to additional well-known difficulties, such as the weight transport problem, where the backward pathway must use the exact transpose of the forward weights (Crick, 1989; Lillicrap et al., 2020). These requirements make standard backpropagation highly difficult to reconcile with known biological learning mechanisms.

This does not reduce the importance of backpropagation as an artificial learning algorithm. It remains powerful precisely because it computes the gradient that perturbation-based methods can only estimate. However, its biological implausibility motivates the search for alternative learning rules that use more biologically realistic ingredients. This is the motivation for the biologically plausible alternatives considered next.

2.3 Biologically plausible alternatives to backpropagation

The biological implausibility of strict backpropagation has motivated a broad range of alternative learning rules. These methods differ in how closely they try to approximate backpropagation and in which biological constraints they prioritize. Some aim to preserve much of the efficiency of backpropagation while relaxing specific assumptions, such as exact symmetric feedback weights. Others avoid explicit backward error propagation altogether, instead relying on local objectives, recurrent dynamics, or stochastic exploration. The purpose of this section is not to review all such methods in detail, but to briefly place perturbation-based learning within the broader landscape of biologically plausible alternatives.

Feedback alignment changes how error information is sent to hidden layers. In standard backpropagation, the error signal is propagated backward using the exact transpose of the forward weight matrices. Feedback alignment replaces these exact backward weights with fixed random feedback weights. Surprisingly, the network can still learn, because the forward weights adapt so that the random feedback signals become useful for reducing the loss (Lillicrap et al., 2016). Direct feedback alignment takes this idea further by sending the output error directly to each hidden layer through fixed random feedback connections, rather than propagating the error recursively from layer to layer (Nøkland, 2016). These methods are biologically relevant because they relax the weight transport problem: the backward pathway no longer needs to contain an exact copy of the forward weights. However, they still rely on relatively detailed, vector-valued error signals being delivered to hidden layers, and therefore remain closer to backpropagation than to scalar-feedback learning rules.

Predictive coding takes a different approach. The basic idea is that higher layers generate predictions of lower-layer activity, and any mismatch between the prediction and the actual activity becomes a local prediction error (Rao and Ballard, 1999). Learning then changes the connections so that these prediction errors become smaller in the future. Although predictive coding was originally developed as a theory of perception, later work showed that it can also be used for supervised learning in multilayer networks: under certain assumptions, the local prediction errors can approximate the error signals used by backpropagation, allowing the network to learn

input-output mappings through more local Hebbian-like updates (Whittington and Bogacz, 2017). Predictive coding is therefore biologically attractive because it replaces a separate backward gradient pass with local prediction errors and recurrent interaction between layers. However, it is more complex than simple scalar-feedback rules because the network must first settle into a consistent state before the weights are updated.

The forward-forward algorithm removes the backward pass entirely. Instead of computing gradients by sending errors backward through the network, it trains each layer to distinguish between positive and negative examples (Hinton, 2022). A positive example is a real or correct input, such as an image paired with its true label, while a negative example is an incorrect or corrupted input, such as the same image paired with a wrong label. Each layer is trained to produce high activity for positive examples and low activity for negative examples, using a local measure of “goodness” such as the squared activity in that layer. The layers are therefore trained in a coordinated way because they see the same positive and negative cases, but each layer uses its own local goodness objective rather than receiving an error signal propagated from the final output. This makes the method more local than backpropagation, because each layer is trained from its own activity rather than from a global gradient. However, the method also changes the learning problem: performance depends on how the positive and negative examples are constructed, and the method does not directly estimate the gradient of one global supervised loss.

Perturbation methods take a different and conceptually simpler approach. Instead of computing an error derivative for each neuron or synapse, they inject random perturbations into either the weights or the neural activities and observe how these perturbations affect performance. If a perturbation improves performance, the corresponding change is reinforced; if it worsens performance, the change is reversed. This gives learning rules that combine local information about the perturbation and neural activity with a global scalar feedback signal, making them closely related to three-factor learning principles. Weight perturbation applies this idea at the level of synaptic weights, while node perturbation applies it at the level of neural activity (Williams, 1992; Werfel et al., 2003; Hiratani et al., 2022). Perturbation methods are generally less efficient than backpropagation, but they are attractive because they require only forward evaluations, stochastic fluctuations, and scalar feedback.

This thesis focuses on perturbation methods for three reasons. First, they align naturally with the biological principles introduced earlier: local synaptic information, stochastic neural activity or synaptic fluctuations, and global modulatory feedback. Second, they are simple enough to implement and analyze in standard neural network models, while still being plausible candidates for more biologically motivated architectures. Third, the literature contains fewer general theoretical comparisons between different perturbation methods than between backpropagation-like alternatives. Rather than surveying many biologically plausible algorithms superficially, this thesis therefore focuses on one family of methods in depth. The remainder of this chapter reviews weight perturbation, node perturbation, and existing comparisons between them, before identifying the literature gap that motivates the original theoretical contribution in Chapter 3.

Having established why perturbation-based learning is a relevant family of biologically motivated alternatives to backpropagation, we now turn to the research objectives of the thesis. The following sections address Objective 1 by reviewing the main perturbation methods in the literature and identifying what remains unresolved. This establishes the current state of perturbation-based learning and leads naturally to Objective 2, which is addressed in Chapter 3 by building on the literature review to motivate and develop the node perturbation variant proposed in this thesis.

2.4 Weight perturbation

Weight perturbation applies perturbation-based learning directly in parameter space by adding random noise to the network weights and biases and observing how the resulting perturbed network affects the loss. For example, if a weight has value 1.00, a sampled perturbation of 0.07 gives the perturbed weight 1.07. If the resulting forward pass gives a lower loss than some baseline, the update should move the weight in the same direction, for example from 1.00 to 1.03. If the perturbed network performs worse, the update should instead move in the opposite direction. In

this way, random parameter fluctuations are correlated with a scalar learning signal to produce a gradient estimate. This is the basic idea behind weight perturbation, but there are several ways to formalize it. Below, we derive the score-function update rule used in the rest of this thesis. At the end of the subsection, we briefly discuss another common formulation, the finite-difference perturbation rule.

In this section, we formalize weight perturbation using the score-function estimator. This estimator is also known as the likelihood-ratio gradient estimator in the Monte Carlo simulation literature (Glynn, 1995). In reinforcement learning, the same log-derivative principle is closely associated with the REINFORCE algorithm of Williams (1992), which derives unbiased gradient-following rules from stochastic actions and scalar feedback. Although REINFORCE was not introduced as a perturbation-learning method, it provides the estimator structure underlying the multiply-by-perturbation update used here. This update form is also used in much of the recent perturbation-learning literature considered here, including Werfel et al. (2003), Hiratani et al. (2022), and Züge et al. (2023). These works motivate and analyze the rule in different ways, for example through small-perturbation limits, expected gradient alignment, or learning-curve analysis. Here, we derive the update rule explicitly from the score-function identity. This provides a clean way to view weight perturbation as an unbiased Monte Carlo estimator of the gradient of an expected loss under parameter perturbations.

Let $\boldsymbol{\mu}$ denote the deterministic parameter vector that we seek to optimize, consisting of all weights and biases in the network. Instead of minimizing the deterministic loss $L(\boldsymbol{\mu})$ directly, we introduce a distribution over perturbed parameter vectors,

$$\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 I),$$

where $\sigma > 0$ is a fixed perturbation scale. A single draw from this distribution perturbs the full parameter vector at once, meaning that all weights and biases are perturbed simultaneously rather than one parameter at a time. Perturbing one parameter at a time would give a more isolated estimate of each parameter’s effect, but would require one or more additional forward evaluations per parameter and is therefore impractical for large networks. From a biological perspective, this is even clearer: with roughly 10^{14} synapses in the human brain, a one-synapse-at-a-time strategy would be infeasible. The parallel approach introduces more randomness into each update, but greatly reduces the number of required forward passes. This choice is consistent with the broader simultaneous-perturbation literature, most notably simultaneous perturbation stochastic approximation, where all coordinates are perturbed together rather than estimated one at a time (Spall, 1992). The same principle underlies the WP–NP methods considered here. The objective is then the expected loss under this distribution,

$$F(\boldsymbol{\mu}) := \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta})].$$

Here, $F(\boldsymbol{\mu})$ is the expected loss of the perturbed network, $p_{\boldsymbol{\mu}}$ is the Gaussian distribution over parameters centered at the learnable mean $\boldsymbol{\mu}$, and $L(\boldsymbol{\theta})$ is the loss obtained when the sampled parameter vector $\boldsymbol{\theta}$ is used in the forward pass. Minimizing this expectation means choosing $\boldsymbol{\mu}$ so that the distribution $p_{\boldsymbol{\mu}}$ places high probability on parameter values with low loss and low probability on parameter values with high loss. Conceptually, this is similar to ordinary loss minimization, but it is expressed in a stochastic setting where the loss is averaged over random perturbations. In the limit $\sigma \rightarrow 0$, the distribution collapses to a point mass at $\boldsymbol{\mu}$, and the expected loss reduces to the deterministic loss $L(\boldsymbol{\mu})$. We now seek to construct an estimator of $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$ using only forward evaluations of perturbed networks. To compute $\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu})$, we write the expectation explicitly as

$$F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta},$$

and differentiate under the integral sign:

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \int L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) d\boldsymbol{\theta}.$$

This step is valid under standard regularity conditions, and allows us to make the dependence on $\boldsymbol{\mu}$ explicit inside the integral. Following the score-function estimator (Williams, 1992), we use the

identity

$$\nabla_{\boldsymbol{\mu}} p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}),$$

which yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta}) \nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})]. \quad (9)$$

This expression rewrites the gradient of an expectation as an expectation involving the score function $\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta})$, enabling gradient estimation using Monte Carlo sampling.

For the Gaussian distribution $p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{I})$, the log-density is given by

$$\log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = -\frac{1}{2\sigma^2} \|\boldsymbol{\theta} - \boldsymbol{\mu}\|^2 + C,$$

and its gradient is

$$\nabla_{\boldsymbol{\mu}} \log p_{\boldsymbol{\mu}}(\boldsymbol{\theta}) = \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2}.$$

Substituting into (9) yields

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\theta}) \frac{\boldsymbol{\theta} - \boldsymbol{\mu}}{\sigma^2} \right].$$

Using the reparameterization $\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma \boldsymbol{\xi}$, we obtain

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\boldsymbol{\xi}}{\sigma} \right]. \quad (10)$$

In practice, the expectation in (10) cannot be computed exactly, as it requires integrating over all possible perturbations. Instead, we approximate it using Monte Carlo sampling, which allows us to estimate the gradient using only forward evaluations of the network. For a single sample $\boldsymbol{\xi}$, this yields the estimator

$$\hat{g}_i = L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\xi_i}{\sigma}.$$

To reduce the variance of this estimator, one can subtract a baseline b that does not depend on the individual perturbation ξ_i , following the standard baseline argument used in REINFORCE (Williams, 1992). The estimator remains unbiased, since

$$\mathbb{E} \left[b \frac{\xi_i}{\sigma} \right] = 0,$$

as ξ_i has zero mean and b is independent of ξ_i . One example is a leave-one-out batch-mean baseline: if several perturbations are sampled in the same batch, the baseline for one perturbation can be formed from the average loss of the other perturbations. Intuitively, such a baseline centers the learning signal: perturbations that lead to lower-than-average loss reinforce their contribution, while those that perform worse than average are reversed. This reduces variance in the gradient estimate without affecting its expectation. The resulting estimator becomes

$$\hat{g}_i = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}. \quad (11)$$

Applying gradient descent, we update the parameters in the negative gradient direction:

$$\Delta \mu_i = -\eta (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_i}{\sigma}, \quad (12)$$

where η is the learning rate. This corresponds to the weight perturbation update rule. This update rule constitutes an unbiased Monte Carlo estimator of the gradient of the expected loss, implying that its expected update direction coincides with the true backpropagation gradient, although potentially with high variance.

This makes WP biologically plausible in the sense discussed above. The update does not require a neuron-specific error derivative, but only a stochastic perturbation of the parameter and a scalar feedback signal measuring whether the perturbed network performed better or worse than the baseline. The term $L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b$ therefore plays a role similar to the global modulatory signal M in Eq. (1), while ξ_i represents the local stochastic fluctuation being reinforced or suppressed. Thus, WP fits the idea that random fluctuations in synaptic efficacy can be correlated with later reward or error signals to drive learning.

2.5 Node perturbation

Node perturbation provides an alternative to weight perturbation by applying noise at the level of neural activity rather than directly to each individual parameter. Recall that the pre-activation of neuron j in layer ℓ is

$$z_j^\ell = \sum_i W_{ij}^\ell x_i^{\ell-1} + b_j^\ell.$$

Instead of perturbing each incoming weight W_{ij}^ℓ and bias b_j^ℓ separately, node perturbation perturbs z_j^ℓ directly. The resulting scalar learning signal is then assigned back to the incoming weights using local presynaptic activity. The motivation is that neural networks typically contain many more weights than nodes: between a layer with $N_{\ell-1}$ inputs and N_ℓ outputs, weight perturbation requires $N_{\ell-1}N_\ell$ perturbations, whereas node perturbation requires only N_ℓ . Node perturbation can therefore be viewed as a lower-dimensional perturbation method that preserves the same basic ingredients as weight perturbation: stochastic fluctuations, local synaptic information, and a global scalar learning signal.

To derive the standard node perturbation update, we use the same score-function logic as for weight perturbation, but apply it to perturbations of neural pre-activations rather than parameters. Thus, in this section, $\boldsymbol{\mu}$ denotes the deterministic vector of pre-activations to be perturbed, and $\boldsymbol{\theta}$ denotes the corresponding perturbed pre-activation vector:

$$\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 I).$$

Equivalently, using the reparameterization

$$\boldsymbol{\theta} = \boldsymbol{\mu} + \sigma \boldsymbol{\xi}, \quad \boldsymbol{\xi} \sim \mathcal{N}(0, I),$$

node perturbation estimates how changes in the perturbed pre-activations affect the loss.

As in the weight perturbation case, the score-function estimator gives

$$\nabla_{\boldsymbol{\mu}} F(\boldsymbol{\mu}) = \mathbb{E} \left[L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) \frac{\boldsymbol{\xi}}{\sigma} \right],$$

where $F(\boldsymbol{\mu}) = \mathbb{E}_{\boldsymbol{\theta} \sim p_{\boldsymbol{\mu}}} [L(\boldsymbol{\theta})]$ is the expected loss under node perturbations. For a single perturbation sample, and with a baseline b independent of the individual perturbation component ξ_j^ℓ , this gives the pre-activation-level estimator

$$\widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Equivalently, if $\epsilon_j^\ell = \sigma \xi_j^\ell$, then

$$\widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \boldsymbol{\epsilon}) - b) \frac{\epsilon_j^\ell}{\sigma^2}.$$

This estimator gives the gradient estimate with respect to the perturbed pre-activation z_j^ℓ . To update the network parameters, we must translate this pre-activation-level estimate into estimates for the incoming weights and bias.

Using the local derivatives from the pre-activation equation above,

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}, \quad \frac{\partial z_j^\ell}{\partial b_j^\ell} = 1,$$

the estimated weight gradient is

$$\widehat{g}_{W_{ij}^\ell} = \widehat{g}_{z_j^\ell} \frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma} x_i^{\ell-1},$$

and the estimated bias gradient is

$$\widehat{g}_{b_j^\ell} = \widehat{g}_{z_j^\ell} = (L(\boldsymbol{\mu} + \sigma \boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Applying gradient descent gives the standard score-function-style node perturbation updates

$$\Delta W_{ij}^\ell = -\eta(L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma} x_i^{\ell-1},$$

and

$$\Delta b_j^\ell = -\eta(L(\boldsymbol{\mu} + \sigma\boldsymbol{\xi}) - b) \frac{\xi_j^\ell}{\sigma}.$$

Thus, node perturbation has the same basic score-function structure as weight perturbation, but the perturbation is applied to pre-activations and then assigned to incoming weights using local presynaptic activity. Compared to WP, the NP update only adds the presynaptic activity term $x_i^{\ell-1}$, which is locally available to the incoming synapse. The update therefore fits naturally within the three-factor structure in Eq. (1): a local synaptic factor, a stochastic perturbation of neural activity, and a global scalar feedback signal. In this sense, node perturbation is also biologically plausible.

2.6 Other variants of Weight Perturbation and Node Perturbation

The weight perturbation and node perturbation rules derived above are both score-function-style estimators, and they are closely related to many of the perturbation-based methods in the recent literature. However, several other variants of weight and node perturbation have also been proposed. Broadly, these methods differ in where the perturbation is applied, how the perturbation is sampled or scaled, and how the scalar learning signal is formed. Some methods perturb weights directly, others perturb neural activity or pre-activations, while others modify the perturbation scale, baseline, or update assignment.

Since the first research objective of this thesis is to establish the current state of the perturbation-based learning literature, we have systematically mapped the most influential and relevant articles found on this topic. The table was created both to address this research objective and to make sure, as far as possible, that the method derived later in the thesis has not already been proposed in the existing literature. Table 1 collects 21 update rules across 9 articles. Because these articles use different notation, perturb different variables, choose different noise distributions, and define their learning signals in different ways, the table rewrites the methods in a common notation. This makes it possible to compare them directly by perturbed variable, noise sampling, and update rule.

One important distinction shown in Table 1 is between finite-difference methods and score-function-style estimators. The earliest perturbation-based update rules included in the table, such as Madaline Rule III from Widrow and Lehr (1990) and the methods of Flower and Jabri (1992), use finite-difference perturbation rules. In these methods, the update is based on a first-order approximation of how the loss changes under a small perturbation. In other words, they estimate a derivative using the classical idea of dividing a change in loss by the applied perturbation, $\Delta L/\Delta x$. This leads to update rules where the learning signal is divided by the perturbation. By contrast, the score-function-style estimators derived above multiply the scalar learning signal by the sampled perturbation. An early neural-network example of this type of update is given by Cauwenberghs (1992), and the later perturbation-learning papers in Table 1 use the same general score-function-style form, including Werfel et al. (2003), Cho et al. (2011), Hara et al. (2017), Hiratani et al. (2022), and Züge et al. (2023). Because of this, and because the theoretical contribution developed in Chapter 3 is also based on a score-function-style update rule, the rest of this thesis focuses on score-function estimators. Finite-difference variants are nevertheless included in the table because they are historically important and closely related.

Because Table 1 contains 21 different update rules, not all of them can be included in the empirical evaluation. The evaluation therefore focuses on the WP and NP methods derived above, which serve as the baseline perturbation methods for much of the recent literature in the table, together with one existing variant from Werfel et al. (2003). This variant is included because it changes the scale of node perturbations, and is therefore the closest existing competitor to the scaling method developed later in the thesis. The remaining variants modify other aspects of the learning rule, such as the baseline, weight normalization, or decorrelation of the inputs. These ideas could

in principle be combined with the scaling method developed in Chapter 3, rather than replacing it. Since they are not direct competitors to the proposed scaling method, and since they are not widely adopted as standard methods in the later perturbation-learning literature, testing them in combination with the new method is left as future work rather than included in the empirical evaluation. However, the broader perturbation-learning literature remains important for the rest of the thesis: several ideas and results from the other papers, including Züge et al. (2023), Werfel et al. (2003), and Hiratani et al. (2022), become important in the later discussion, starting already in the next section where we compare WP and NP.

For compactness, Table 1 rewrites the notation from the different papers using the notation of this thesis. Perturbations are written as a scale multiplied by unit-scale noise, for example $\delta z = \sigma \xi$ with $\xi \sim \mathcal{N}(0, I)$, unless another scale is explicitly shown. The scalar learning signal is denoted by $\Delta \ell$. In general, $\Delta \ell$ should be understood as the loss from a perturbed forward pass minus some baseline or reference loss. The exact baseline is only written explicitly when it is important for the method itself, as in the noisy-baseline variants of Cho et al. (2011) and Hara et al. (2017). The main focus of the table is on where the perturbation is applied, how the noise is sampled, and how the resulting update is assigned to the parameters, since these are the aspects most relevant to the theoretical and empirical comparisons in this thesis. Let U_k denote the update direction, so that weights are updated as $W_k \leftarrow W_k - \eta_W U_k$. For decorrelated variants, $x_{k-1}^* = R_{k-1} x_{k-1}$, and the decorrelation matrix is updated using

$$R_k \leftarrow R_k - \eta_R (x_k^* (x_k^*)^\top - \text{diag}((x_k^*)^2)) R_k.$$

Method-specific constants that are absorbed into the learning rate in the original papers are kept at the same level of detail as in the source update rule.

Table 1: Comparison of perturbation-based update rules in the literature.

Reference	Method	Perturbed variable	Noise sampling	Update rule
Widrow and Lehr (1990)	Madaline Rule III	Pre-activation	$\delta z_i = \sigma \xi_i$; no distribution specified	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_i} x_j$
Flower and Jabri (1992)	Neuron perturbation	Pre-activation	$\delta z_i = \sigma \xi_i, \xi_i \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_i} x_j$
Flower and Jabri (1992)	Weight perturbation	Weight	$\delta W_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_{ij}}$
Flower and Jabri (1992)	Summed weight neuron perturbation	Incoming weights	$\delta W_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \text{Unif}\{-1, 1\}$	$U_{ij} = \frac{\Delta \ell}{\sigma \xi_{ij}}, r_i = \sum_m \sigma \xi_{im} x_m$
Cauwenberghs (1992)	Stochastic error descent	Parameter	$\delta p_i = \sigma \xi_i, \mathbb{E}[\xi_i \xi_j] = \delta_{ij}$	$U_i = \Delta \ell \sigma \xi_i$
Werfel et al. (2003)	Node perturbation	Output	$\delta y_i = \sqrt{N} \sigma \xi_i, \xi_i \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sqrt{N} \sigma} \xi_i x_j$
Werfel et al. (2003)	Weight perturbation	Weight	$\delta W_{ij} = \sigma \xi_{ij}, \xi_{ij} \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sigma} \xi_{ij}$
Cho et al. (2011)	Node perturbation	Output	$\delta y_i = \sigma \xi_i, \xi_i \sim \mathcal{N}(0, 1)$	$U_{ij} = \Delta \ell \sigma \xi_i x_j, \Delta \ell = L_\xi - L$
Cho et al. (2011)	Noisy-baseline node perturbation	Output	$\delta y_i = \sigma \xi_i, \delta y'_i = \sigma \zeta_i, \xi_i, \zeta_i \sim \mathcal{N}(0, 1)$	$U_{ij} = \Delta \ell \sigma \xi_i x_j, \Delta \ell = L_\xi - L_\zeta$
Hara et al. (2017)	Single node perturbation	Output	$\delta y_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, 1)$	$U_{jk} = \frac{\Delta \ell}{N \sigma} \xi_k x_j, \Delta \ell = L_\xi - L$
Hara et al. (2017)	Double node perturbation	Output	$\delta y_k = \sigma \xi_k, \delta y'_k = \sigma \zeta_k, \xi_k, \zeta_k \sim \mathcal{N}(0, 1)$	$U_{jk} = \frac{\Delta \ell}{N \sigma} \xi_k x_j, \Delta \ell = L_\xi - L_\zeta$
Hiratani et al. (2022)	Node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Hiratani et al. (2022)	Weight-normalized node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top, w_i^k \leftarrow \frac{\ w_i^k\ }{\ w_i^k - \eta_W u_i^k\ } (w_i^k - \eta_W u_i^k)$
Züge et al. (2023)	Weight perturbation	Weight	$\delta W_{ij} = \sigma_{\text{WP}} \xi_{ij}^{\text{WP}}, \xi_{ij}^{\text{WP}} \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sigma_{\text{WP}}} \xi_{ij}^{\text{WP}}, \sigma_{\text{WP}} = \frac{\sigma_{\text{eff}}}{\alpha \sqrt{N_{\text{eff}}}}$
Züge et al. (2023)	Node perturbation	Pre-activation	$\delta z_{it} = \sigma_{\text{eff}} \xi_{it}^{\text{NP}}, \xi_{it}^{\text{NP}} \sim \mathcal{N}(0, 1)$	$U_{ij} = \frac{\Delta \ell}{\sigma_{\text{eff}}} \sum_{t=1}^T \xi_{it}^{\text{NP}} x_{jt}$
Dalm et al. (2024)	Node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k x_{k-1}^\top$
Dalm et al. (2024)	Iterative node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N_k} \Delta \ell \frac{\xi_k}{\ \xi\ _2^2} x_{k-1}^\top$
Dalm et al. (2024)	Activity-based node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\ \delta a\ _2^2} x_{k-1}^\top$
Dalm et al. (2024)	Decorrelated node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \frac{\Delta \ell}{\sigma} \xi_k (x_{k-1}^*)^\top$
Dalm et al. (2024)	Decorrelated iterative node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N_k} \Delta \ell \frac{\xi_k}{\ \xi\ _2^2} (x_{k-1}^*)^\top$
Dalm et al. (2024)	Decorrelated activity-based node perturbation	Pre-activation	$\delta z_k = \sigma \xi_k, \xi_k \sim \mathcal{N}(0, I_k)$	$U_k = \sqrt{N} \Delta \ell \frac{\delta a_k}{\ \delta a\ _2^2} (x_{k-1}^*)^\top$

2.7 Existing comparisons of weight and node perturbation

Weight perturbation and node perturbation are closely related learning rules: both replace explicit gradient computation with random perturbations and a scalar feedback signal. This makes it natural to ask which method should be expected to learn more efficiently. The standard intuition is based on the dimensionality of the perturbation space. Weight perturbation injects noise into individual parameters, whereas node perturbation injects noise into neuron activities and then assigns the resulting learning signal to incoming weights using local presynaptic activity. Since neural networks typically contain many more weights than nodes, node perturbation is often expected to produce less noisy updates and faster learning. However, this dimensionality argument is only an intuition, not a theoretical proof. The existing literature contains a small number of more direct comparisons between weight and node perturbation, which we review in this section.

An early comparison appears in the finite-difference setting studied by Flower and Jabri (1992). Their discussion presents standard weight perturbation as computationally more expensive than node perturbation, because weight perturbation estimates the effect of perturbing individual weights rather than nodes. This provides an early version of the idea that perturbing nodes can be more efficient than perturbing individual weights, although the comparison is made in a finite-difference framework rather than in the score-function-style setting considered in the rest of this thesis.

The most direct analytical comparison between WP and NP is given by Werfel et al. (2003). They analyze a linear perceptron with N inputs and M outputs, comparing direct gradient descent, node perturbation, and weight perturbation. In this setting, NP perturbs the M -dimensional output vector, whereas WP perturbs the full MN -dimensional weight matrix. Their learning-curve analysis shows that, when the scalar update parameter is chosen optimally, NP is slower than direct gradient descent by a factor equal to the number of output units M , while WP is slower than NP by an additional factor equal to the number of input units N . This provides a precise setting in which the lower perturbation dimension of NP leads to faster learning than WP. However, the result is derived for a simplified linear network with a specific input distribution, target function, and objective, and the authors explicitly note that the conclusions are not universally applicable.

Züge et al. (2023) provide an important caveat to the usual dimensionality argument. Their analysis concerns temporally extended tasks, where a trial consists of multiple time steps before a scalar feedback signal is received. This is different from the standard feedforward setting considered above, where each input produces one forward pass and one loss value. In temporally extended tasks, the relevant perturbation dimension is more ambiguous: WP perturbs the weights once per trial, while NP may need to perturb neural activity across all time steps. As a result, the effective perturbation space explored by NP can grow with the duration of the trial, and may no longer be smaller than the space explored by WP. Their results therefore show that WP can perform similarly to, or better than, NP in some temporally extended tasks. This does not invalidate the standard argument that NP is often more efficient in feedforward settings, but it shows that the advantage is not universal and depends on the structure of the learning problem.

Later perturbation-learning papers reflect the same interpretation of the WP–NP relationship, often by referring back to the same core results discussed above. Lillicrap et al. (2020) describe weight perturbation as a simple scalar-feedback method, but state that it can be improved by perturbing neuron outputs rather than individual weights, referring to early node perturbation work such as Flower and Jabri (1992). Hiratani et al. (2022) make the dimensionality argument more explicit: they describe WP as noisier than NP in the vanilla setting, citing Werfel et al. (2003), while also noting that WP can be competitive with, or better than, NP when the feedback signal is sparse, citing Züge et al. (2023). Thus, later work appears to rely on the same basic picture: NP is often expected to be more efficient because it perturbs a lower-dimensional space, but this expectation is qualified by known exceptions where WP can be competitive or preferable.

2.8 Literature gap

The literature reviewed above provides a clear intuition for why node perturbation is often expected to be more efficient than weight perturbation: it perturbs a lower-dimensional space. This intuition is supported by analytical results in simplified linear settings and is commonly used in later work to motivate node perturbation over weight perturbation. However, the comparison remains incomplete. The strongest analytical result is derived for linear perceptrons, while later work has pointed to temporally extended tasks as cases where the standard dimensionality argument does not directly apply. Although these temporally extended settings are not the focus of this thesis, their role in the literature is important: they show that the relationship between WP and NP is still treated as conditional and context-dependent, rather than theoretically resolved. Thus, although node perturbation is often presented as the more efficient default, the literature does not provide a general account of the relationship between weight perturbation and node perturbation in nonlinear multilayer networks.

This is the gap addressed in Chapter 3. Rather than asking only which method performs better empirically, we analyse how the two estimators are related theoretically in multilayer networks. As shown there, this analysis not only clarifies the relationship between weight-level and node-level perturbations, but also motivates a new variant of node perturbation that is designed to better match the perturbations induced by weight perturbation. The key idea used to make this comparison is introduced in the next section, which reviews the local reparameterization trick from variational inference in Bayesian neural networks.

2.9 The local reparameterization trick

The literature gap identified above concerns the relationship between weight-level and node-level perturbations. To approach this gap, we turn to an idea from a seemingly different context: the local reparameterization trick of Kingma et al. (2015). Kingma et al. study stochastic variational inference in Bayesian neural networks, not perturbation-based learning. However, their work addresses a closely related structural question: when noise is sampled at the level of weights, can its effect instead be represented by equivalent noise at the level of pre-activations? In their setting, they show that this change of perspective reduces the variance of the gradient estimator with respect to the posterior variance parameter σ_{ij}^2 . This subsection reviews their setup and derivation, with particular attention to the similarities and differences between their setting and the WP–NP setting considered in this thesis.

In an ordinary neural network, training produces a single value for each weight. A Bayesian neural network instead treats the weights as random variables. Rather than learning one fixed parameter vector, the goal is to learn a distribution over possible parameter values. This is useful because the distribution can represent uncertainty about the weights, rather than only a single best estimate. Exact Bayesian inference is usually intractable in neural networks, so variational inference introduces a simpler approximate posterior distribution over the weights, denoted $q_\phi(w)$, and optimizes its parameters ϕ .

For example, Kingma et al. consider a fully factorized Gaussian approximate posterior over the weights,

$$q_\phi(w_{ij}) = \mathcal{N}(\mu_{ij}, \sigma_{ij}^2), \quad (13)$$

where the variational parameters are the means μ_{ij} and variances σ_{ij}^2 . Training then means adjusting these parameters so that the approximate posterior explains the data while remaining close to a prior distribution over weights. This is done by maximizing the variational lower bound

$$\mathcal{L}(\phi) = -D_{\text{KL}}(q_\phi(w) \| p(w)) + \mathcal{L}_D(\phi), \quad (14)$$

where the expected log-likelihood term is

$$\mathcal{L}_D(\phi) = \sum_{(x,y) \in D} \mathbb{E}_{q_\phi(w)} [\log p(y \mid x, w)]. \quad (15)$$

The expectation over $q_\phi(w)$ is generally not available in closed form, since it would require integrating over all possible weight values. Kingma et al. therefore use Monte Carlo sampling: weights are sampled from the approximate posterior, the network is evaluated using these sampled weights, and gradients with respect to μ_{ij} and σ_{ij}^2 are computed by backpropagation through the sampled network.

For a Gaussian posterior, this sampling can be written using the reparameterization

$$w_{ij} = \mu_{ij} + \sigma_{ij}\xi_{w,ij}, \quad \xi_{w,ij} \sim \mathcal{N}(0, 1). \quad (16)$$

Thus, the stochasticity is introduced at the weight level. In this sense, the setting has a superficial similarity to weight perturbation: random noise is added to weights in order to form a stochastic estimate of a gradient. The difference is that in stochastic variational inference the gradient is still computed by backpropagation through the sampled network, whereas perturbation-based learning uses scalar feedback and does not backpropagate error derivatives.

Kingma et al. observe that the sampled weights influence the loss through intermediate quantities such as the pre-activations. Consider one linear layer, where $x_{m,i}$ is the activation of input unit i for datapoint m , and

$$z_{m,j} = \sum_i x_{m,i} w_{ij}. \quad (17)$$

Since $z_{m,j}$ is a linear combination of Gaussian random variables, it is also Gaussian. Substituting the reparameterized weights gives

$$z_{m,j} = \sum_i x_{m,i} \mu_{ij} + \sum_i x_{m,i} \sigma_{ij} \xi_{w,ij}. \quad (18)$$

The first term is deterministic given the input, while the second term is the pre-activation noise induced by the sampled weights. We therefore write

$$z_{m,j} = \gamma_{m,j} + \Delta z_{m,j}, \quad (19)$$

where

$$\gamma_{m,j} = \sum_i x_{m,i} \mu_{ij}, \quad \Delta z_{m,j} = \sum_i x_{m,i} \sigma_{ij} \xi_{w,ij}. \quad (20)$$

Because the $\xi_{w,ij}$ are independent standard Gaussian variables, the induced pre-activation noise is Gaussian:

$$\Delta z_{m,j} \sim \mathcal{N}(0, \delta_{m,j}), \quad (21)$$

with

$$\delta_{m,j} = \sum_i x_{m,i}^2 \sigma_{ij}^2. \quad (22)$$

Thus,

$$z_{m,j} \sim \mathcal{N}(\gamma_{m,j}, \delta_{m,j}). \quad (23)$$

The local reparameterization trick is to sample this pre-activation directly:

$$z_{m,j} = \gamma_{m,j} + \sqrt{\delta_{m,j}} \xi_{z,m,j}, \quad \xi_{z,m,j} \sim \mathcal{N}(0, 1), \quad (24)$$

instead of sampling each individual weight w_{ij} and then computing $z_{m,j}$. Equivalently, the induced pre-activation noise is sampled as

$$\Delta z_{m,j} = \sqrt{\delta_{m,j}} \xi_{z,m,j}. \quad (25)$$

This replaces global weight noise with local pre-activation noise. Kingma et al. then show that the locally reparameterized estimator has lower variance than the estimator based on direct weight sampling. Their argument is given for the gradient with respect to the posterior variance parameter σ_{ij}^2 . For simplicity, consider a minibatch of size one, and condition on a single pre-activation $z_{m,j}$.

For both estimators, the chain rule gives

$$\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} = \frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \frac{\partial z_{m,j}}{\partial \sigma_{ij}^2}. \quad (26)$$

For direct weight sampling, using the weight reparameterization in Eq. (16) and the pre-activation definition in Eq. (17), the second factor is obtained as follows. Since

$$\frac{\partial w_{ij}}{\partial \sigma_{ij}^2} = \frac{\xi_{w,ij}}{2\sigma_{ij}}, \quad (27)$$

we get

$$\frac{\partial z_{m,j}}{\partial \sigma_{ij}^2} = \frac{x_{m,i}\xi_{w,ij}}{2\sigma_{ij}}, \quad (28)$$

and therefore

$$\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} = \frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \frac{x_{m,i}\xi_{w,ij}}{2\sigma_{ij}}. \quad (29)$$

For the locally reparameterized estimator, the same chain-rule expression in Eq. (26) applies, but the second factor changes. Using the local pre-activation sampling defined above, and since

$$\frac{\partial \delta_{m,j}}{\partial \sigma_{ij}^2} = x_{m,i}^2, \quad (30)$$

we obtain

$$\frac{\partial z_{m,j}}{\partial \sigma_{ij}^2} = \xi_{z,m,j} \frac{x_{m,i}^2}{2\sqrt{\delta_{m,j}}}, \quad (31)$$

and therefore

$$\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} = \frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \frac{\xi_{z,m,j} x_{m,i}^2}{2\sqrt{\delta_{m,j}}}. \quad (32)$$

The two estimators have the same expectation, but different variance. To compare them, Kingma et al. condition on the sampled pre-activation $z_{m,j}$ and apply the law of total variance:

$$\text{Var}_{q_\phi, D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} \right] = \text{Var}_{z_{m,j}} \left[\mathbb{E}_{q_\phi, D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} \mid z_{m,j} \right] \right] + \mathbb{E}_{z_{m,j}} \left[\text{Var}_{q_\phi, D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} \mid z_{m,j} \right] \right]. \quad (33)$$

The first term is identical for the two estimators, because both estimators induce the same distribution over $z_{m,j}$. The difference lies in the second term, which measures how much randomness remains in the gradient estimator after the pre-activation has been fixed.

For direct weight sampling, the conditional variance can be decomposed into two terms. This follows from applying the variance rule for a product of conditionally independent random quantities. After conditioning on $z_{m,j}$, the gradient term $\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}}$ may still be random because of the data sample and the remaining perturbations in the network. The individual weight-noise variable $\xi_{w,ij}$ may also still be random because $z_{m,j}$ does not uniquely determine $\xi_{w,ij}$. Therefore,

$$\begin{aligned} \mathbb{E}_{z_{m,j}} \left[\text{Var}_{q_\phi, D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} \mid z_{m,j} \right] \right] &= \mathbb{E}_{z_{m,j}} \left[\text{Var}_{q_\phi, D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \mid z_{m,j} \right] q \right] \\ &+ \mathbb{E}_{z_{m,j}} \left[\mathbb{E}_{q_\phi, D} \left[\left(\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \right)^2 \mid z_{m,j} \right] \text{Var}_{q_\phi, D} [\xi_{w,ij} \mid z_{m,j}] \frac{x_{m,i}^2}{4\sigma_{ij}^2} \right], \end{aligned}$$

where

$$q = \frac{(z_{m,j} - \gamma_{m,j})^2 x_{m,i}^4}{4\delta_{m,j}}.$$

The first term is the variance remaining in the loss derivative after conditioning on $z_{m,j}$. The second term is the additional variance caused by the fact that $\xi_{w,ij}$ can still vary even when $z_{m,j}$ is fixed.

For the locally reparameterized estimator, we can decompose the corresponding conditional variance in the same way. The first term is identical, since the distribution of $z_{m,j}$ and the conditional

randomness in the loss derivative are the same. The difference is in the second term. Under local reparameterization, conditioning on $z_{m,j}$ uniquely determines the local noise variable,

$$\xi_{z,m,j} = \frac{z_{m,j} - \gamma_{m,j}}{\sqrt{\delta_{m,j}}}.$$

Thus,

$$\text{Var}_{q_{\phi},D} [\xi_{z,m,j} \mid z_{m,j}] = 0.$$

Consequently, the second term vanishes, and the conditional variance reduces to

$$\mathbb{E}_{z_{m,j}} \left[\text{Var}_{q_{\phi},D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial \sigma_{ij}^2} \mid z_{m,j} \right] \right] = \mathbb{E}_{z_{m,j}} \left[\text{Var}_{q_{\phi},D} \left[\frac{\partial \mathcal{L}_D^{\text{SGVB}}}{\partial z_{m,j}} \mid z_{m,j} \right] q \right].$$

The locally reparameterized estimator therefore removes the additional non-negative variance term that appears under direct weight sampling.

The intuition is that the forward pass depends on the sampled weights only through the pre-activations they produce. Under direct weight sampling, many different weight-noise configurations can lead to the same pre-activation $z_{m,j}$, and therefore to the same forward computation. However, the gradient estimator still depends on the individual weight-noise variable $\xi_{w,ij}$, so these different configurations can produce different updates even though they produce the same network output. This adds variance that is unrelated to any change in the learning signal. By sampling the induced pre-activation noise directly, the locally reparameterized estimator removes this unnecessary weight-level randomness.

Whether the same local reparameterization idea can be used to compare WP and NP is not obvious. The setting above differs from the perturbation-learning setting in several important ways. Kingma et al. consider stochastic variational inference in Bayesian neural networks, where the weights have learned posterior means and variances. Their variance-reduction result is derived for a pathwise gradient estimator with respect to the posterior variance parameter σ_{ij}^2 , and the gradient is computed by backpropagation through the sampled network. In the WP setting considered in this thesis, the perturbation scale is fixed, the parameters being updated are the mean weights, and the gradient estimate is obtained from scalar feedback using a score-function estimator rather than backpropagation.

At the same time, the similarity is clear enough to make the question natural. In both cases, a neural network is evaluated after sampling noise at the level of weights, and the resulting stochastic computation is used to estimate a gradient. The local reparameterization trick shows that, in one setting, weight-level noise can be replaced by equivalent pre-activation-level noise in a way that reduces estimator variance. The question is whether the same logic can be applied to the WP–NP setting. This is what Chapter 3 investigates.

2.10 Summary of the Theory

The first part of this chapter set the background for the thesis by briefly introducing biological learning, learning in artificial neural networks, why backpropagation is biologically implausible, and some biologically plausible alternatives to backpropagation. The rest of the chapter then addressed the first research objective by examining what has been proposed and theoretically established in the literature on perturbation-based learning. At a high level, this meant clarifying what perturbation-based learning is, what the main variants in the literature are, how they relate to each other, and what remains unresolved. We did this by deriving the two main score-function-style perturbation families considered in the literature, weight perturbation and node perturbation, and by mapping related variants in a common notation in Table 1. We then reviewed existing comparisons between weight perturbation and node perturbation, showing that the relationship between the two families remains incomplete. Finally, we introduced the local reparameterization trick from a different literature as a potentially useful idea for resolving this gap. The next chapter builds on this foundation by relating weight and node perturbation through induced noise.

3 Relating Weight and Node Perturbation Through Induced Noise

This chapter addresses the second research objective of the thesis by building on the existing literature in order to clarify the relationship between common perturbation algorithms and use this understanding to develop a theoretically motivated variant of node perturbation. Chapter 2 identified an unresolved gap in the perturbation-learning literature: although WP and NP are often compared, their estimator-level relationship in nonlinear multilayer networks has not been fully characterized. Chapter 2 also introduced the local reparameterization trick of Kingma et al. (2015), where weight-level noise is translated into equivalent pre-activation-level noise in the context of stochastic variational inference for Bayesian neural networks. Kingma et al. show that this reduces gradient-estimator variance. The natural question is therefore whether WP has the structure needed for a similar local reparameterization argument to be applied in the perturbation-learning setting.

This chapter shows that the answer is yes. Gaussian WP induces a specific noise distribution at the level of pre-activations. By sampling this induced pre-activation noise directly, we obtain a new node perturbation rule, input-scaled node perturbation (IS-NP). We then show that WP and IS-NP estimate the same perturbation-smoothed objective in expectation, and that IS-NP has lower or equal coordinate-wise estimator variance than WP. The general variance-reduction logic follows the local reparameterization argument of Kingma et al. (2015). However, the application of this logic to perturbation-based learning has, to the best of our knowledge, not previously been done in the literature. This includes the derivation of the WP-induced pre-activation noise in this setting, the construction of IS-NP from that induced noise, the expected-equivalence result between WP and IS-NP, and the corresponding variance comparison. The remainder of this chapter presents our novel contributions to the literature.

3.1 Pre-activation noise induced by WP

To apply the local reparameterization idea to perturbation-based learning, we need to know whether the effect of WP can be described directly at the level of pre-activations. In other words, if weight perturbations induce a well-defined distribution over pre-activation perturbations, then we can sample from that distribution directly instead of sampling perturbations at the weight level. This is the key step that makes it possible to compare WP with a node-level perturbation rule. We therefore begin by deriving the distribution of the pre-activation noise induced by Gaussian weight perturbations. Recall from Section 2.4 that we modeled perturbations as a deterministic variable plus a scale times unit Gaussian noise. For neural networks, this corresponds to perturbing each weight and bias independently as

$$\mathbf{W}^{\ell'} = \mathbf{W}^{\ell} + \sigma \boldsymbol{\xi}_W^{\ell}, \quad \mathbf{b}^{\ell'} = \mathbf{b}^{\ell} + \sigma \boldsymbol{\xi}_b^{\ell}, \quad \xi_{W,ij}^{\ell}, \xi_{b,j}^{\ell} \sim \mathcal{N}(0, 1) \text{ i.i.d.},$$

where $\xi_{W,ij}^{\ell}$ denotes the (i, j) -entry of the unit-noise matrix $\boldsymbol{\xi}_W^{\ell}$, and $\xi_{b,j}^{\ell}$ denotes the unit-noise perturbation of bias b_j^{ℓ} .

The pre-activation at neuron j in layer ℓ is given by

$$z_j^{\ell} = \sum_i W_{ij}^{\ell} x_i^{\ell-1} + b_j^{\ell}.$$

After perturbation, this becomes

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell'} x_i^{\ell-1'} + b_j^{\ell'} = \sum_i (W_{ij}^{\ell} + \sigma \xi_{W,ij}^{\ell}) x_i^{\ell-1'} + b_j^{\ell} + \sigma \xi_{b,j}^{\ell}.$$

This expression separates into a deterministic pre-activation component and a stochastic component induced by the weight and bias perturbations:

$$z_j^{\ell'} = \sum_i W_{ij}^{\ell} x_i^{\ell-1'} + b_j^{\ell} + \sigma \sum_i \xi_{W,ij}^{\ell} x_i^{\ell-1'} + \sigma \xi_{b,j}^{\ell}.$$

Define the deterministic component as

$$\gamma_j^\ell := \sum_i W_{ij}^\ell x_i^{\ell-1'} + b_j^\ell. \quad (34)$$

Define the induced pre-activation noise as

$$\Delta z_j^\ell := \sigma \sum_i \xi_{W,ij}^\ell x_i^{\ell-1'} + \sigma \xi_{b,j}^\ell. \quad (35)$$

We can now write the full pre-activation in compact form as

$$z_j^{\ell'} = \gamma_j^\ell + \Delta z_j^\ell. \quad (36)$$

Δz_j^ℓ is the effect of weight perturbation at the pre-activation level that we wanted to express. We now want to characterize the distribution of this noise term.

We start by conditioning on the incoming perturbed activations $\mathbf{x}^{\ell-1'}$. Under this conditioning, the factors $x_i^{\ell-1'}$ are fixed, while the weight- and bias-noise variables $\xi_{W,ij}^\ell$ and $\xi_{b,j}^\ell$ remain independent standard Gaussians. Therefore each weight-noise term $\sigma x_i^{\ell-1'} \xi_{W,ij}^\ell$ is Gaussian:

$$\sigma x_i^{\ell-1'} \xi_{W,ij}^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}(0, \sigma^2 (x_i^{\ell-1'})^2). \quad (37)$$

Since Δz_j^ℓ is a sum of independent Gaussian random variables, it is also Gaussian. Its conditional mean is

$$\mathbb{E}[\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'}] = \sigma \sum_i x_i^{\ell-1'} \mathbb{E}[\xi_{W,ij}^\ell] = 0, \quad (38)$$

and its conditional variance is

$$\text{Var}(\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'}) = \sigma^2 \sum_i (x_i^{\ell-1'})^2 \text{Var}(\xi_{W,ij}^\ell) + \sigma^2 \text{Var}(\xi_{b,j}^\ell) = \sigma^2 (1 + \|\mathbf{x}^{\ell-1'}\|^2). \quad (39)$$

Thus,

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}(0, \delta_j^\ell), \quad (40)$$

with

$$\delta_j^\ell = \sigma^2 (1 + \|\mathbf{x}^{\ell-1'}\|^2). \quad (41)$$

This mirrors the key observation behind the local reparameterization trick in Section 2.9: independent Gaussian noise at the weight level induces Gaussian noise at the pre-activation level. The variance expression is not identical, since Kingma et al. (2015) consider learned weight-specific variances, while WP uses a fixed perturbation scale σ , with an additional contribution from the bias perturbation. For the argument here, however, the important point is simply that the induced pre-activation noise has a well-defined Gaussian distribution. This means that the same pre-activation noise can be sampled directly at the node level, which is the starting point for the method derived next.

3.2 Input-scaled node perturbation

The result above suggests a natural modification of standard node perturbation. Recall from Section 2.5 that standard NP injects Gaussian noise with the same perturbation scale at every node. From this point onward, we refer to this standard version as vanilla NP. Although the perturbation scale is a hyperparameter, it is fixed across neurons and does not depend on the magnitude of the incoming activation vector. This means that vanilla NP does not reproduce the pre-activation noise induced by WP: as shown in Section 3.1, Gaussian weight and bias perturbations induce node-level noise whose variance scales with $1 + \|\mathbf{x}^{\ell-1'}\|^2$. It is therefore not possible to apply the local reparameterization trick directly to vanilla NP and obtain a clean variance comparison with WP.

One way to approximate this input-dependent scaling is fan-in-scaled NP. If the incoming activations are independent standard Gaussians, $x_i^{\ell-1} \sim \mathcal{N}(0, 1)$, then

$$\mathbb{E}\left[1 + \|\mathbf{x}^{\ell-1}\|^2\right] = \mathbb{E}\left[1 + \sum_{i=1}^{N_{\ell-1}} (x_i^{\ell-1})^2\right] = N_{\ell-1} + 1. \quad (42)$$

Under this assumption, the induced WP noise has expected variance $\sigma^2(N_{\ell-1} + 1)$ at each node when the bias is included. This is the same type of scaling argument used by Werfel et al. (2003), who choose the NP noise variance to be a factor N larger than the WP noise variance in order to make the two stochastic variants directly comparable. However, fan-in-scaled NP only matches the WP-induced node noise under this distributional assumption. In general, there is no guarantee that the incoming activations in every layer, for every input, and throughout training have squared norm $N_{\ell-1}$. Unless such a normalization is imposed explicitly, fan-in scaling only matches the typical scale of the WP-induced noise, not the actual pre-activation noise in a given forward pass. It therefore also cannot be used as the direct local reparameterized version of WP.

We now remove this expectation-level approximation by matching the WP-induced pre-activation noise directly. This gives a node-level perturbation rule that samples from the same pre-activation noise distribution as WP, and can therefore be viewed as the local reparameterized version of WP in the perturbation-learning setting. We call this method input-scaled node perturbation (IS-NP). For each neuron j in layer ℓ , we sample

$$\Delta z_j^\ell = \sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2} \xi_{z,j}^\ell, \quad \xi_{z,j}^\ell \sim \mathcal{N}(0, 1). \quad (43)$$

This gives the same conditional distribution as the WP-induced pre-activation noise derived in Section 3.1:

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1}\|^2\right)\right). \quad (44)$$

We now derive the corresponding update rule. Since the perturbation standard deviation is $\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}$, the score-function term for the pre-activation perturbation is

$$\frac{\Delta z_j^\ell}{\sigma^2 \left(1 + \|\mathbf{x}^{\ell-1}\|^2\right)} = \frac{\xi_{z,j}^\ell}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}}. \quad (45)$$

The resulting gradient estimate with respect to the pre-activation is therefore

$$\hat{g}_{z_j^\ell} = \Delta \ell \frac{\xi_{z,j}^\ell}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}}. \quad (46)$$

Using the local derivatives

$$\frac{\partial z_j^\ell}{\partial W_{ij}^\ell} = x_i^{\ell-1}, \quad \frac{\partial z_j^\ell}{\partial b_j^\ell} = 1, \quad (47)$$

we obtain the corresponding weight-gradient estimator

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta \ell \frac{\xi_{z,j}^\ell x_i^{\ell-1}}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}}. \quad (48)$$

The corresponding bias-gradient estimator is

$$\hat{g}_{b_j^\ell}^{\text{IS-NP}} = \Delta \ell \frac{\xi_{z,j}^\ell}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}}. \quad (49)$$

Applying gradient descent gives the update rules

$$\Delta W_{ij}^\ell = -\eta \Delta \ell \frac{\xi_{z,j}^\ell x_i^{\ell-1}}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1}\|^2}}. \quad (50)$$

and

$$\Delta b_j^\ell = -\eta \Delta \ell \frac{\xi_{z,j}^\ell}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (51)$$

Thus, unlike vanilla NP, the perturbation scale is not fixed across parameters. Unlike fan-in-scaled NP, it is also not determined only by the number of incoming units. Instead, IS-NP scales the perturbation by the actual norm of the incoming activation vector for each layer and input example. This does not by itself imply that IS-NP is preferable to other NP variants in practice, but it makes IS-NP directly comparable to WP because it samples the same pre-activation noise distribution induced by Gaussian weight perturbations.

In terms of biological plausibility, IS-NP remains close to the three-factor learning picture. Compared to standard NP, it also uses the norm of the full incoming activation vector together with the bias contribution, $\sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}$, rather than only the activity of the single presynaptic neuron connected to a given synapse. This goes beyond the simplest form of Eq. (1), but the added information is still based on presynaptic activity arriving at the postsynaptic neuron, together with the node perturbation and the global scalar feedback signal. IS-NP therefore remains well aligned with the general ingredients of three-factor learning, while assuming an additional neuron-level scaling term compared to standard NP.

IS-NP and fan-in-scaled NP are also unbiased estimators of the gradient of the loss averaged over their own perturbation distributions. The unbiasedness argument is not derived separately here, since it follows the same score-function proof as for standard NP; only the perturbation distribution is changed. When σ is very small, these smoothed losses approach the original loss, so the estimators (WP, IS-NP, fan-in NP, and vanilla NP) all target nearly the same objective. For finite σ , however, the objectives need not be exactly identical. Appendix C shows that WP and IS-NP are a special case: because IS-NP samples the same pre-activation noise distribution induced by WP, the two methods estimate the gradient of exactly the same smoothed objective. This is not generally true for vanilla NP or fan-in-scaled NP. This exact equality in the smoothed objective makes the subsequent variance comparison between WP and IS-NP clean.

Just as the locally reparameterized estimator in Eq. (32) samples the activation-level noise induced by direct weight sampling in Eq. (29), IS-NP samples the pre-activation noise induced by WP directly at the node level. We now use this correspondence to ask whether the same variance-reduction logic applies: does replacing WP with its local reparameterized version, IS-NP, reduce estimator variance?

3.3 Variance reduction theorem

We now show that IS-NP can be viewed as the conditional expectation of WP given the induced pre-activation noise field. This means that IS-NP removes weight-noise variability that affects the WP update but does not affect the actual perturbed forward pass. The result is a direct variance reduction by the law of total variance.

For a single weight coordinate W_{ij}^ℓ , the WP estimator can be written as

$$\hat{g}_{ij,\ell}^{\text{WP}} = \Delta \ell \frac{\xi_{W,ij}^\ell}{\sigma}. \quad (52)$$

The corresponding bias-coordinate estimator is

$$\hat{g}_{b_j^\ell}^{\text{WP}} = \Delta \ell \frac{\xi_{b,j}^\ell}{\sigma}. \quad (53)$$

Theorem 1. *For Gaussian weight and bias perturbations and the corresponding input-scaled node perturbation estimator defined in Eqs. (48) and (49), the coordinate-wise variance of IS-NP is no larger than that of WP. For each weight coordinate,*

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) \leq \text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}).$$

The same inequality holds for each bias coordinate b_j^ℓ .

Proof. We consider the weight perturbation estimator in (52), and analyze its variance by conditioning on the induced pre-activation noise. Recall from (35) that Δz_j^ℓ is a linear combination of the weight perturbations.

We condition on the full induced noise field $\Delta \mathbf{Z} = (\Delta \mathbf{z}^1, \dots, \Delta \mathbf{z}^L)$. Since the perturbed forward pass satisfies

$$\mathbf{z}^{\ell'} = \mathbf{W}^\ell \mathbf{x}^{\ell-1'} + \mathbf{b}^\ell + \Delta \mathbf{z}^\ell,$$

the entire perturbed trajectory $(\mathbf{x}^{1'}, \dots, \mathbf{x}^{L'})$ is uniquely determined by $\Delta \mathbf{Z}$. Hence both the scalar learning signal $\Delta \ell(\Delta \mathbf{Z})$ and all activations $\mathbf{x}^{\ell'}$ are deterministic given $\Delta \mathbf{Z}$.

Now fix (i, j, ℓ) . Given $\Delta \mathbf{Z}$, the incoming activation vector $\mathbf{x}^{\ell-1'}$ is fixed, and the only component of the induced noise field that contains information about $\xi_{W,ij}^\ell$ is Δz_j^ℓ . From (35), Δz_j^ℓ is a linear combination of Gaussian weight and bias perturbations. Hence, $\xi_{W,ij}^\ell$ and Δz_j^ℓ are jointly Gaussian. Since both variables have mean zero, the conditional expectation is linear and given by

$$\mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \frac{\text{Cov}(\xi_{W,ij}^\ell, \Delta z_j^\ell)}{\text{Var}(\Delta z_j^\ell)} \Delta z_j^\ell. \quad (54)$$

The covariance term is

$$\text{Cov}(\xi_{W,ij}^\ell, \Delta z_j^\ell) = \text{Cov}\left(\xi_{W,ij}^\ell, \sigma \sum_k \xi_{W,kj}^\ell x_k^{\ell-1'} + \sigma \xi_{b,j}^\ell\right) \quad (55)$$

$$= \sigma \sum_k x_k^{\ell-1'} \text{Cov}(\xi_{W,ij}^\ell, \xi_{W,kj}^\ell) \quad (56)$$

$$= \sigma x_i^{\ell-1'}, \quad (57)$$

where the last equality follows from the independence and unit variance of the Gaussian weight perturbations. Similarly,

$$\text{Var}(\Delta z_j^\ell) = \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right). \quad (58)$$

Substituting these expressions gives

$$\mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \frac{x_i^{\ell-1'}}{\sigma \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)} \Delta z_j^\ell. \quad (59)$$

Therefore,

$$\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z}] = \frac{\Delta \ell(\Delta \mathbf{Z})}{\sigma} \mathbb{E}[\xi_{W,ij}^\ell \mid \Delta \mathbf{Z}] = \Delta \ell(\Delta \mathbf{Z}) \frac{x_i^{\ell-1'} \Delta z_j^\ell}{\sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)}. \quad (60)$$

Recall from (48) that the IS-NP estimator is

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta \ell \frac{\xi_{z,j}^\ell x_i^{\ell-1'}}{\sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2}}. \quad (61)$$

Since the IS-NP perturbation satisfies

$$\Delta z_j^\ell = \sigma \sqrt{1 + \|\mathbf{x}^{\ell-1'}\|^2} \xi_{z,j}^\ell, \quad (62)$$

the same estimator can equivalently be written as

$$\hat{g}_{ij,\ell}^{\text{IS-NP}} = \Delta \ell \frac{x_i^{\ell-1'} \Delta z_j^\ell}{\sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)}. \quad (63)$$

Hence,

$$\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z}] = \hat{g}_{ij,\ell}^{\text{IS-NP}}. \quad (64)$$

The law of total variance states that for random variables Y and X ,

$$\text{Var}(Y) = \text{Var}(\mathbb{E}[Y \mid X]) + \mathbb{E}[\text{Var}(Y \mid X)].$$

Applying this identity with $Y = \hat{g}_{ij,\ell}^{\text{WP}}$ and $X = \Delta \mathbf{Z}$,

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}) = \text{Var}(\mathbb{E}[\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z}]) + \mathbb{E}[\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z})].$$

Using the identity above,

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}) = \text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) + \mathbb{E}[\text{Var}(\hat{g}_{ij,\ell}^{\text{WP}} \mid \Delta \mathbf{Z})].$$

Since the second term is nonnegative, it follows that

$$\text{Var}(\hat{g}_{ij,\ell}^{\text{IS-NP}}) \leq \text{Var}(\hat{g}_{ij,\ell}^{\text{WP}}).$$

The proof for the bias coordinate is identical, with the constant bias input taking the place of $x_i^{\ell-1'}$. In that case,

$$\text{Cov}(\xi_{b,j}^\ell, \Delta z_j^\ell) = \sigma,$$

which gives

$$\mathbb{E}[\xi_{b,j}^\ell \mid \Delta \mathbf{Z}] = \frac{\Delta z_j^\ell}{\sigma(1 + \|\mathbf{x}^{\ell-1'}\|^2)}.$$

Substituting this into the WP bias estimator in Eq. (53) gives the IS-NP bias estimator in Eq. (49), so the same law-of-total-variance argument applies. $\square$

The theorem confirms that the same variance-reduction principle does carry over to perturbation-based learning: IS-NP is the local reparameterized version of WP, and its coordinate-wise variance is smaller than or equal to that of WP. The intuition is the same as in the variational Bayesian neural network setting discussed in Section 2.9. Once the induced pre-activation noise is fixed, the perturbed forward pass and the scalar learning signal are fixed; any remaining weight-level randomness only changes the WP update, not the network output. IS-NP removes this unnecessary source of variability by sampling the induced pre-activation noise directly.

3.4 Why IS-NP may improve over existing NP variants

The variance theorem above compares IS-NP directly with WP, but it does not establish how IS-NP compares to other node perturbation variants. As shown in Appendix C, NP and fan-in-scaled NP use different perturbation distributions, and therefore generally correspond to perturbation-smoothed objectives that are similar, but not identical, to those of WP and IS-NP. For this reason, we do not derive an analytical variance comparison between IS-NP and these methods. Instead, we use the scaling arguments developed above to reason about which perturbation rule is likely to be better behaved in practice, before testing this empirically in Chapter 5.

Standard NP uses the same perturbation scale at every node, regardless of the incoming activation norm. This can lead to poorly scaled updates. If $\|\mathbf{x}^{\ell-1}\|$ is large, a fixed pre-activation perturbation may have only a small relative effect on the network output, while the resulting weight update can still become large because it is multiplied by the incoming activation. Conversely, if $\|\mathbf{x}^{\ell-1}\|$ is small, the same fixed perturbation may create a large relative change in the pre-activation, while the corresponding weight update is small. In both cases, the perturbation scale is disconnected from the activation weight scale that determines the weight update. This mismatch is expected to be most problematic in networks where activation norms vary substantially across layers, samples, or training time, for example when layer widths differ or activations are not explicitly normalized. In

such cases, a fixed perturbation scale will sometimes be too large and sometimes too small, making standard NP less consistently scaled than methods that account for the incoming activation norm.

Fan-in-scaled NP partially addresses this issue by accounting for the number of incoming units. Under the independent, zero-mean, unit-variance activation assumption with the bias treated as an additional constant input, Eq. (42) shows that the expected squared norm of the incoming activation vector, including the bias contribution, grows with the fan-in. This is the same scaling argument used by Werfel et al. (2003), who choose the NP perturbation variance to be a factor N larger than the WP perturbation variance so that the two stochastic variants are directly comparable in their linear setting. If the incoming activations were explicitly normalized so that their squared norm matched the layer width, this correction would be exact up to the details of the normalization. However, this would introduce an additional architectural mechanism that is not part of the perturbation-learning methods reviewed here. Some reviewed work uses related stabilization or preprocessing techniques, such as the weight-normalized NP variant of Hiratani et al. (2022), but these do not correspond to layer-wise activation normalization that fixes $1 + \|\mathbf{x}^{\ell-1}\|^2$ in each forward pass. Moreover, common artificial normalization methods such as layer normalization typically include learnable scale and shift parameters, so they do not generally imply a fixed activation norm unless constrained to do so. Relying on such normalization would therefore make fan-in scaling depend on an extra assumption, and one whose biological interpretation is not established in the perturbation-learning literature considered here. In the setting studied in this thesis, activation norms are therefore allowed to vary across layers, samples, and training time. Fan-in scaling can therefore improve on standard NP by correcting for the typical size of a layer, but it still does not account for the actual incoming activation norm in a given forward pass.

IS-NP avoids this average correction by using the actual incoming activation norm, together with the bias contribution, in the current forward pass. Its perturbation scale is therefore adapted to the layer, input example, and current network state. This does not prove that IS-NP must outperform standard NP or fan-in-scaled NP in every practical setting, since other factors may also influence learning performance. However, it gives a clear scaling-based reason to expect more consistent behaviour when activation norms vary across layers, samples, or training time. Whether this advantage translates into better empirical performance is therefore a question to be tested experimentally.

3.5 Summary of theoretical contribution

This chapter addressed the second research objective by further developing perturbation-based learning. Two main contributions came out of this chapter.

The first is a more precise comparison between weight-level and node-level perturbation learning in nonlinear multilayer networks. By deriving IS-NP as the local reparameterized version of WP, showing that WP and IS-NP estimate the same perturbation-smoothed objective, and proving that IS-NP has lower or equal coordinate-wise variance than WP, we have clarified the estimator-level relationship between the two perturbation families. This addresses the gap identified in Chapter 2, where the existing literature was shown to lack a general comparison of WP and NP in nonlinear multilayer networks.

The second contribution is the derivation of input-scaled node perturbation itself. IS-NP modifies standard NP by scaling the node perturbation according to the norm of the incoming activation vector, including the bias contribution. This makes IS-NP match the pre-activation noise distribution induced by Gaussian WP, rather than using a fixed perturbation scale as in vanilla NP or an average fan-in correction as in fan-in-scaled NP. We also compared IS-NP conceptually with these existing NP variants and gave scaling-based reasons to expect IS-NP to behave more consistently when activation norms vary across layers, samples, or training time. This comparison is not a formal variance proof against vanilla NP or fan-in-scaled NP, but it motivates IS-NP as a theoretically grounded variant of node perturbation.

To the best of our knowledge, neither the exact estimator-level variance comparison between WP and a matching node-level perturbation rule, nor the specific IS-NP update rule derived here, has

previously been presented in the perturbation-learning literature.

Chapters 2 and 3 therefore establish the theoretical foundation for the thesis. Chapter 2 addressed the first research objective by reviewing and clarifying the existing theory of perturbation-based learning, while Chapter 3 addressed the second objective. What remains is to evaluate how WP, standard NP, fan-in-scaled NP, and IS-NP perform in practice. The next chapter therefore turns from theory to experiments, where these four learning rules are compared empirically across the selected benchmark tasks.

4 Method

The goal of the evaluation presented in this chapter is to understand the practical usefulness of perturbation-based learning methods by evaluating them empirically. This directly addresses the third research objective of the thesis, but it also supports the second objective by testing whether the theoretical contribution developed in Chapter 3 leads to practical improvements. Although the theoretical analysis in Chapters 2 and 3 gives useful predictions about the relationship between backpropagation and perturbation methods, and about the relationship between different perturbation methods, theory alone does not determine practical learning performance. Neural network optimization problems are generally high-dimensional and non-convex, and the behaviour of a learning rule depends on more than its expected update or estimator variance in isolation. This evaluation is divided into three related parts.

The first part asks how well perturbation-based learning methods work in practice as alternatives to backpropagation. Backpropagation is included as the standard reference method because it computes the exact gradient that the perturbation methods attempt to estimate. The goal is not to show that perturbation methods outperform backpropagation, but to understand how close they can get, how difficult a task they can solve, and where their performance begins to break down. Therefore, worse performance than backpropagation should not be interpreted as a failure of the empirical evaluation. It is an expected and informative part of the comparison, because it shows the practical cost of replacing exact gradient computation with stochastic perturbation-based estimates. This is important in relation to the motivation of Chapter 1: if backpropagation is difficult or impossible to use in biologically inspired architectures or other constrained settings, it is useful to know how capable perturbation-based learning methods are as substitutes.

The second and third parts evaluate the practical consequences of the two main theoretical contributions from Chapter 3. The first contribution was the variance comparison between WP and IS-NP. Empirically, this raises the question of whether the lower estimator variance of IS-NP leads to better learning performance, faster convergence, or more stable updates than WP in practice. The second contribution was the derivation of IS-NP as a new node perturbation rule with input-dependent scaling. Empirically, this raises the question of whether scaling perturbations by the actual incoming activation norm improves over the fixed perturbation scale of vanilla NP or the layer-width-based correction of fan-in-scaled NP. The empirical evaluation outlined here tests whether the theoretical advantages identified in Chapter 3 translate into practical improvements.

To answer these questions, the methods are evaluated on four supervised learning problems of increasing complexity: synthetic sinus regression, California housing regression, MNIST classification, and CIFAR-10 classification. This progression is chosen so that the experiments can show not only whether a method works, but also how its performance changes as the learning problem becomes more difficult. Simple tasks reveal whether the methods can learn under favourable conditions, while harder tasks make the differences between learning rules more visible. The purpose of including difficult tasks is both to clearly reveal differences between methods and to identify where the perturbation methods become inefficient, unstable, or unable to match the reference performance of backpropagation.

All code used to produce the results in this thesis is available in the GitHub repository <https://github.com/MatsStuben/backprop-alternatives>. This includes the implementation of the learning algorithms, model training, metric calculations, hyperparameter tuning, and generation of plots.

4.1 Learning problems and datasets

The empirical evaluation uses four supervised learning problems of increasing complexity. Three of these are standard datasets commonly used in machine learning experiments: California housing, MNIST, and CIFAR-10. The fourth is a synthetic sinus regression task generated specifically to provide a simple and controlled nonlinear regression problem. Together, these tasks span low-dimensional synthetic regression, real tabular regression, simple image classification, and more difficult image classification. The purpose is not to maximize benchmark performance on any one dataset, but to compare how the learning rules behave as the problem becomes more challenging.

The preprocessing is kept simple and standard. The imported datasets are used in their usual form, with only the transformations needed to make them suitable for the fully connected models used in the experiments. This includes standardization for the tabular regression task and pixel scaling, mean-centering, flattening, and one-hot label encoding for the image classification tasks. The following sections describe each learning problem before summarizing the dataset properties in Table 2.

4.1.1 Synthetic sinus regression

The sinus task is a synthetic regression problem designed to provide a controlled nonlinear learning problem. The input is an eight-dimensional vector, and the target is generated from a smooth nonlinear function containing sinusoidal terms, quadratic terms, and a small interaction term. This makes the task more interesting than a linear regression problem while still being simple enough that all learning methods should be expected to make progress.

For an input vector $\mathbf{x} \in [-1, 1]^8$, the noiseless target function is

$$f(\mathbf{x}) = \frac{1}{\sqrt{8}} \left(0.6 \sum_{i=1}^8 \sin(\pi x_i) + 0.3 \sum_{i=1}^8 x_i^2 + 0.2 x_1 x_2 \right).$$

The dataset consists of 512 training examples and 512 test examples. The task is a scalar regression problem, so each input vector is mapped to one real-valued output. The training inputs are sampled uniformly from $[-1, 1]^8$. Training targets are corrupted with additive Gaussian noise with standard deviation 0.1, while the test targets are generated from the noiseless target function. The dataset is generated directly in the experiment.

No external preprocessing is applied to the sinus inputs in the main experiment. The model receives the sampled input vector directly, so the input values remain in the range $[-1, 1]$.

4.1.2 California housing regression

The California housing task is a real tabular regression problem. The goal is to predict median house value from housing and geographical features describing districts in California. Compared with the sinus task, this is a less controlled problem because the data come from a real dataset rather than a generated function, but it remains low-dimensional and is therefore a useful intermediate regression task.

The dataset contains 20,640 examples. It is split into 16,512 training examples and 4,128 test examples, corresponding to an 80/20 train-test split. Each input has eight features, and the output is one real-valued target, so the task is scalar regression. The dataset is loaded using `sklearn.datasets.fetch_california_housing`.

The input features are standardized using the mean and standard deviation computed from the training split. The target values are standardized in the same way. The test set is transformed using the training-split statistics, so no information from the test set is used during preprocessing.

4.1.3 MNIST classification

MNIST is a handwritten digit classification task. Each example is a grayscale image of a digit from 0 to 9, and the goal is to assign the image to the correct digit class. The task is substantially higher-dimensional than the two regression tasks, but the images are simple and centered, making MNIST a standard first benchmark for image classification methods.

The full MNIST dataset contains 60,000 training images and 10,000 test images, and the full training and test sets are used in this thesis. Each image has size 28×28 , which gives 784 input values after flattening. The output is a 10-dimensional vector corresponding to the ten digit classes, and the task is classification. The dataset is loaded using `torchvision.datasets.MNIST`.

Pixel values are scaled to $[0, 1]$, mean-centered using the scalar mean of the training set, and flattened into vectors. The class labels are converted to one-hot vectors for training.

4.1.4 CIFAR-10 classification

CIFAR-10 is an image classification task with ten object classes. Each example is a small colour image, and the goal is to assign the image to the correct class. Compared with MNIST, CIFAR-10 is substantially more difficult because the images contain colour, background variation, and more complex visual structure. It is therefore included as the hardest task in the empirical evaluation.

The full CIFAR-10 dataset contains 50,000 training images and 10,000 test images, and the full training and test sets are used in this thesis. Each image has size 32×32 with three colour channels, giving 3072 input values after flattening. The output is a 10-dimensional vector corresponding to the ten classes, and the task is classification. The dataset is loaded using `torchvision.datasets.CIFAR10`.

Pixel values are scaled to $[0, 1]$, mean-centered using the scalar mean of the training set, rearranged to channel-first format, and flattened into vectors. The class labels are converted to one-hot vectors for training.

4.1.5 Scaled-input diagnostics

In addition to the four main learning problems, a scaled-input diagnostic experiment is performed on the sinus task. In the main sinus experiment, inputs lie in $[-1, 1]^8$. In the scaled-input version, the same underlying target function is used, but the model input is multiplied by 5, giving inputs in $[-5, 5]^8$. The target values are not changed. This isolates the effect of increasing the input norm while keeping the learning problem itself otherwise fixed.

The purpose of this diagnostic is to test how sensitive the perturbation estimators are to changes in activation scale. Vanilla NP uses a fixed node perturbation scale, while fan-in NP scales only by layer width. Both methods therefore ignore the actual norm of the incoming activation vector. IS-NP, in contrast, scales the perturbation using the incoming activation norm, and WP naturally produces pre-activation perturbations whose scale depends on the input. The scaled sinus setting is artificial in the sense that one would normally avoid unnecessary input rescaling, but it provides a clean way to test what happens when the scale assumptions behind fixed or layer-width-based node perturbation are violated.

This is especially useful because the sinus architecture uses equal hidden-layer widths, so the diagnostic is not confounded by different layer sizes. The experiment therefore focuses directly on the effect of input scale. The same kind of scale mismatch could also occur inside deeper networks if activation norms vary strongly across layers or samples, but the sinus diagnostic makes the effect visible in a simple controlled setting.

Since the scaled-input experiment is designed to test estimator quality under input rescaling, we report cosine similarity and estimator variance rather than learning curves. The diagnostics use the same setup as the main sinus task in Table 4: 10 random seeds and checkpoints at epochs 1,

120, and 240 of the BP diagnostic run. The meaning of these seeds and checkpoints is explained later in this chapter.

Table 2: Datasets used in the empirical evaluation.

Property	Synthetic sinus	California housing	MNIST	CIFAR-10
Task type	Regression	Regression	Classification	Classification
Input $\rightarrow$ output	$8 \rightarrow 1$	$8 \rightarrow 1$	$784 \rightarrow 10$	$3072 \rightarrow 10$
Train size	512	16,512	60,000	50,000
Test size	512	4,128	10,000	10,000
Score	R^2	R^2	Accuracy	Accuracy
Difficulty	1	2	3	4

4.2 Model architecture and initialization

All experiments use fully connected multilayer perceptrons (MLPs). This choice is made to keep the empirical evaluation aligned with the theoretical setting considered in the thesis and to isolate differences between learning rules rather than differences between architectures. For image classification tasks such as MNIST and CIFAR-10, convolutional networks would normally be a more appropriate architecture and would likely give better absolute performance. However, the goal here is not to maximize benchmark accuracy, but to compare perturbation-based learning rules under a common model class. This is also consistent with much of the perturbation-learning literature, where fully connected networks are commonly used to study the behaviour of the learning rule itself. The hidden-layer activation function is sigmoid for the two regression tasks and ReLU for the two classification tasks, while all output layers are linear. The specific layer sizes used for each task are shown in Table 3. All layers are implemented as PyTorch `nn.Linear` layers with bias terms. Since no custom initialization is applied, the weights and biases use the default PyTorch initialization for linear layers, which is Kaiming-uniform.

Table 3: Model architectures used in the empirical evaluation.

Property	Sinus regression	California housing	MNIST	CIFAR-10
Layer sizes	8-8-8-1	8-128-64-1	784-256-128-10	3072-512-256-10
Hidden activation	Sigmoid	Sigmoid	ReLU	ReLU
Output activation	Linear	Linear	Linear	Linear
Biases	Yes	Yes	Yes	Yes
Initialization	Kaiming-uniform	Kaiming-uniform	Kaiming-uniform	Kaiming-uniform

4.3 Algorithms

The perturbation estimators used in the experiments are the same score-function-style estimators derived in Chapters 2 and 3. This section therefore does not rederive the learning rules, but specifies the implementation choices used in the empirical evaluation. These choices are important because they determine how the theoretical estimators are turned into practical mini-batch training algorithms.

All learning methods are trained using mini-batches. The task-specific mini-batch sizes are given in Table 4. For backpropagation, this is the standard mini-batch SGD update: the MSE loss is averaged over the mini-batch, the gradient is computed by backpropagation, and the parameters are updated with SGD. No momentum, Adam, weight decay, or learning-rate schedule is used. This choice was made to keep the comparison focused on the learning rules themselves. The perturbation methods are also evaluated in their basic form, without additional optimizer mechanisms or regularization techniques. Such extensions could be interesting in future work, but the aim here is first to compare the vanilla versions of the methods under the same simple SGD-style update setting. Backpropagation is therefore not listed as a separate algorithm below, since it is the standard SGD implementation of the gradient computation described in Chapter 2. It is included only as the reference method against which the perturbation methods are compared.

For the perturbation methods, independent perturbations are sampled for each example in the mini-batch. Thus, each example produces its own noisy forward pass and its own scalar loss. The parameter update is then obtained by averaging the resulting per-example update estimates across the mini-batch. Sampling independent perturbations per example is important because the batch-mean baseline relies on variation between perturbations inside the same mini-batch. If the same perturbation were shared by the whole batch, subtracting the batch mean would remove the useful scalar learning signal.

Let $\mathcal{B} = \{(x_i, y_i)\}_{i=1}^m$ denote a mini-batch of size m , and let ℓ_i be the noisy loss obtained for example i . The mini-batch baseline used in the implementation is

$$b_{\mathcal{B}} = \frac{1}{m} \sum_{j=1}^m \ell_j.$$

The perturbation estimator is then formed using $\ell_i - b_{\mathcal{B}}$. Thus, perturbations that lead to lower-than-average loss give a negative estimated gradient contribution and are reinforced by the gradient-descent update, while perturbations that lead to higher-than-average loss are reversed.

In the theoretical derivation, an independent baseline is required for the usual unbiasedness argument. A leave-one-out mini-batch baseline,

$$b_{-i} = \frac{1}{m-1} \sum_{j \neq i} \ell_j,$$

would satisfy this condition because the baseline for example i would not include the noisy loss generated by the perturbation of that same example. In the implementation, however, the full mini-batch mean $b_{\mathcal{B}}$ is used instead. This means that the baseline is not strictly independent of the perturbation for example i , so the estimator is not exactly the same unbiased estimator as the leave-one-out version. This choice is made for computational reasons: the full batch mean is obtained directly from the losses already computed in the noisy forward pass, while a leave-one-out baseline would require a separate baseline value for every example in the mini-batch. Since the self-inclusion effect is of order $1/m$, it is expected to be small for the batch sizes used in the experiments, and the practical benefit of avoiding the additional computation is considered to outweigh this small approximation.

Algorithm 1 Weight perturbation (WP)

Require: Training set $\mathcal{D}$, model parameters $\theta = \{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i}^W \sim \mathcal{N}(0, I)$ ,  $\xi_{k,i}^b \sim \mathcal{N}(0, I)$ 
8:          $z_{k,i} \leftarrow (W_k + \sigma \xi_{k,i}^W) a_{k-1,i} + b_k + \sigma \xi_{k,i}^b$ 
9:          $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
10:      end for
11:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
12:    end for
13:     $b \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j$ 
14:     $\xi_i \leftarrow \{\xi_{k,i}^W, \xi_{k,i}^b\}_{k=1}^L$  for all  $i$ 
15:     $\theta \leftarrow \theta - \eta \frac{1}{m} \sum_{i=1}^m (\ell_i - b) \xi_i / (\sigma + \epsilon_{\text{num}})$ 
16:  end for
17: end for
```

Algorithm 2 Vanilla node perturbation with fixed pre-activation noise

Require: Training set $\mathcal{D}$, model parameters $\theta = \{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ , set  $s_{k,i} \leftarrow \sigma$ 
8:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
9:          $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
10:      end for
11:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
12:    end for
13:     $b \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j$ 
14:     $\hat{g}_{z,k,i} \leftarrow (\ell_i - b) \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $k, i$ 
15:     $W_k \leftarrow W_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i} a_{k-1,i}^\top$  for all  $k$ 
16:     $b_k \leftarrow b_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i}$  for all  $k$ 
17:  end for
18: end for
```

Algorithm 3 Fan-in scaled node perturbation

Require: Training set $\mathcal{D}$, model parameters $\theta = \{W_k, b_k\}_{k=1}^L$, layer input dimensions d_{k-1} , epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ 
8:          $s_{k,i} \leftarrow \sigma \sqrt{d_{k-1} + 1}$   $\triangleright$  includes the bias input
9:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
10:         $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
11:      end for
12:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
13:    end for
14:     $b \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j$ 
15:     $\hat{g}_{z,k,i} \leftarrow (\ell_i - b) \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $k, i$ 
16:     $W_k \leftarrow W_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i} a_{k-1,i}^\top$  for all  $k$ 
17:     $b_k \leftarrow b_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i}$  for all  $k$ 
18:  end for
19: end for
```

Algorithm 4 Input-scaled node perturbation (IS-NP)

Require: Training set $\mathcal{D}$, model parameters $\theta = \{W_k, b_k\}_{k=1}^L$, epochs E , mini-batch size B , learning rate η , noise scale σ

```
1: for  $e = 1, \dots, E$  do
2:   for mini-batch  $\mathcal{B} \subset \mathcal{D}$  do
3:      $m \leftarrow |\mathcal{B}|$ 
4:     for sample  $i = 1, \dots, m$  do
5:        $a_{0,i} \leftarrow x_i$ 
6:       for layer  $k = 1, \dots, L$  do
7:         Draw  $\xi_{k,i} \sim \mathcal{N}(0, I)$ 
8:          $s_{k,i} \leftarrow \sigma \sqrt{1 + \|a_{k-1,i}\|_2^2}$   $\triangleright$  WP-induced pre-activation scale
9:          $z_{k,i} \leftarrow W_k a_{k-1,i} + b_k + s_{k,i} \xi_{k,i}$ 
10:         $a_{k,i} \leftarrow \phi_k(z_{k,i})$ 
11:      end for
12:       $\ell_i \leftarrow \mathcal{L}(a_{L,i}, y_i)$ 
13:    end for
14:     $b \leftarrow \frac{1}{m} \sum_{j=1}^m \ell_j$ 
15:     $\hat{g}_{z,k,i} \leftarrow (\ell_i - b) \xi_{k,i} / (s_{k,i} + \epsilon_{\text{num}})$  for all  $k, i$ 
16:     $W_k \leftarrow W_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i} a_{k-1,i}^\top$  for all  $k$ 
17:     $b_k \leftarrow b_k - \eta \frac{1}{m} \sum_{i=1}^m \hat{g}_{z,k,i}$  for all  $k$ 
18:  end for
19: end for
```

4.4 Evaluation metrics

In supervised machine learning, the central question is how well a trained model performs on unseen data. The most important empirical quantity is therefore the test loss, since this measures whether the learned function generalizes beyond the examples used for optimization. Training loss is also important, because it shows whether the learning rule is able to reduce the objective on the data it is trained on. These two quantities therefore form the main performance measures in the empirical evaluation.

However, training and test loss provide limited diagnostic information. They show how well a method works, but not why it works or fails. A perturbation-based method may perform poorly because its estimated updates are poorly aligned with the true descent direction, or because the estimates fluctuate strongly between perturbation samples. To better understand the learning curves, we therefore also measure directional alignment and estimator variance. These are natural diagnostics for perturbation-based learning, and they are especially relevant in this thesis because they connect directly to the theoretical results in Chapter 3: the expectation-matching result concerns the average update direction, while the variance theorem predicts a difference in estimator variability. The following sections describe the three metric groups used in the experiments, why they matter, and how they are measured in this thesis.

Table 4: Training and estimator-diagnostic settings for each learning problem.

Task	Train batch	Eval batch	Run epochs	Seeds	Diagnostic checkpoints	Gradient estimates per mini-batch
Sinus regression	64	512	500	10	Epochs 1, 120, and 240 of a 240-epoch BP diagnostic run.	240
California housing	256	4096	200	5	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	200
MNIST	128	1024	400	5	Epochs 1, 10, and 20 of a 20-epoch BP diagnostic run.	100
CIFAR-10	128	1024	80	5	Epochs 1, 25, and 50 of a 50-epoch BP diagnostic run.	10

4.4.1 Training and test performance

In all experiments, the loss is the mean squared error (MSE) between the network prediction and the target. For the regression tasks, this is the squared difference between the predicted scalar output and the scalar target, averaged over examples. For these tasks, training and test score are also reported using the coefficient of determination, R^2 , which measures how much of the target variance is explained by the model. For the classification tasks, the class labels are represented as one-hot vectors, and the MSE is computed between the 10-dimensional network output and the one-hot target vector. Classification performance is additionally evaluated using accuracy, where the predicted class is taken to be the output component with the largest value.

The MSE loss is presented as learning curves showing both training and test loss as functions of epoch. For the classification tasks, training and test accuracy are plotted in the same way. These curves are computed by running each method across multiple random seeds and averaging the result at each epoch, so that the reported trends are not dominated by a single favourable or unfavourable initialization. The number of seeds depends on the size of the task and network, since the larger image experiments are more computationally expensive; the exact number of seeds, training epochs, evaluation batch sizes, and diagnostic settings are shown in Table 4. To summarize the learning curves numerically, we report the best value reached during training for each seed. The summary tables then give the mean $\pm$ standard deviation of these best values across seeds; for example, the reported best test loss is the average of the best test loss reached in each individual seed. To avoid overwhelming the main results with too many numerical columns, the tables in Chapter 5 report only the most important summary metrics: best test loss, best test score, cosine

similarity, and estimator variance. Full summary tables, including training loss, training score, and convergence epoch, are provided in Appendix B. The convergence epoch is defined as the first epoch at which the test loss has achieved 90% of the total improvement from the initial test loss to the best test loss observed during training.

These metrics are interpreted in three main ways. First, the best training and test values show the final performance reached by each method. Second, the convergence epoch indicates how quickly the method approaches its best observed performance. Third, the shape of the learning curves gives information about stability: smooth curves indicate stable learning dynamics, while large fluctuations or spikes suggest that the updates are noisy or unstable. Together, these quantities show not only how well each method performs, but also how efficiently and reliably it learns.

4.4.2 Directional alignment

All perturbation-based update rules considered in this thesis are intended to estimate a useful gradient direction from stochastic perturbations and a scalar learning signal. In expectation, these estimators point in the same direction as the gradient of the objective they estimate. This makes it natural to ask how well a finite-sample perturbation estimate aligns with the true gradient in practice. Intuitively, if the estimated update direction is well aligned with the true gradient, the learning rule is likely to reduce the loss efficiently. Conversely, if the estimated gradient frequently points in incorrect directions, learning becomes inefficient and may even increase the loss. A natural way to quantify this alignment is through the cosine similarity between the estimated gradient $\hat{g}$ and the true gradient g :

$$\cos(\hat{g}, g) = \frac{\hat{g}^\top g}{\|\hat{g}\| \|g\|}.$$

Here, $\hat{g}$ denotes the gradient estimator produced by the learning rule, while $g = \nabla_\theta L(\theta)$ denotes the true gradient, computed using backpropagation in the experiments. The metric takes values in $[-1, 1]$, where 1 indicates perfect alignment, 0 indicates orthogonality, and negative values indicate that the estimated update direction points against the true descent direction.

The relevance of directional alignment can be seen from a first-order approximation of the loss change. Let $L(\theta)$ be a differentiable loss function and consider an update

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta\hat{g},$$

where $\eta > 0$ is the learning rate. A first-order Taylor expansion around θ gives

$$L(\theta^+) = L(\theta + \Delta\theta) \approx L(\theta) + \nabla_\theta L(\theta)^\top \Delta\theta.$$

Writing $g = \nabla_\theta L(\theta)$ and substituting $\Delta\theta = -\eta\hat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^\top \hat{g}.$$

Using

$$g^\top \hat{g} = \|g\| \|\hat{g}\| \cos(\hat{g}, g),$$

this becomes

$$\Delta L \approx -\eta \|g\| \|\hat{g}\| \cos(\hat{g}, g).$$

Thus, for fixed gradient norms, a higher cosine similarity gives a larger first-order decrease in the loss. Positive cosine similarity corresponds to a locally descending update, zero cosine similarity gives no first-order improvement, and negative cosine similarity corresponds to local ascent. This makes cosine similarity a useful diagnostic for the directional quality of perturbation-based gradient estimates, although the actual loss change also depends on update magnitude, estimator variance, and curvature.

Concretely, for each task, a model is first trained with backpropagation and saved at three checkpoints representing early, middle, and later training; the checkpoint epochs are given in Table 4. Backpropagation is used for this diagnostic so that all perturbation rules are compared at the same

parameter values. This makes the comparison between estimators fair: differences in cosine similarity then reflect differences in the update estimates themselves, rather than differences in where each method happens to be during training. Using three checkpoints also tests whether estimator quality changes between the beginning, middle, and later stages of training. At each checkpoint and for each mini-batch in the training set, the reference direction is the exact backpropagation descent direction

$$u_{\text{BP}} = -\nabla_{\theta} L_{\mathcal{B}}(\theta),$$

flattened over all weights and biases. For each perturbation method, the same frozen parameters and the same mini-batch are used to compute K independent gradient estimates. Each estimate is obtained by drawing new random perturbations and applying the corresponding perturbation update rule to the mini-batch, without changing the model parameters. Thus, for method m , this gives K raw update directions

$$\hat{u}_m^{(1)}, \dots, \hat{u}_m^{(K)}.$$

These are then averaged before computing cosine similarity:

$$\bar{u}_m = \frac{1}{K} \sum_{k=1}^K \hat{u}_m^{(k)}.$$

Averaging over K independent perturbation estimates reduces the dependence of the diagnostic on any single perturbation draw. This makes the cosine similarity a more stable and fair comparison of the typical update direction produced by each method.

$$\cos(\bar{u}_m, u_{\text{BP}}) = \frac{\bar{u}_m^{\top} u_{\text{BP}}}{\|\bar{u}_m\| \|u_{\text{BP}}\|}.$$

This value is averaged over all mini-batches in one full pass through the training set and over all random seeds, using 10 seeds for the sinus task and 5 seeds for the other tasks because of their higher computational cost. In the main results, the three checkpoint values are also averaged to give one final cosine value per method and dataset. The summary tables report the mean $\pm$ standard deviation of this aggregate cosine across seeds. The checkpoint-specific cosine values are also saved and plotted separately, but they are moved to the appendix because they do not change the main interpretation and would otherwise add many extra plots to the results chapter.

4.4.3 Estimator variance

Directional alignment measures whether a gradient estimator points in a useful direction, but it does not capture how much the estimate fluctuates between perturbation samples. In perturbation-based learning, the gradient estimate is obtained from random perturbations, so two perturbation samples at the same parameter values can produce different update directions. Even if the estimator is well aligned with the true gradient on average, large fluctuations can make learning inefficient. Variance is also central to the theoretical contribution of the thesis, where IS-NP was shown to have lower or equal coordinate-wise variance than WP under the matched perturbation construction. Estimator variance is therefore a natural diagnostic to measure empirically. Before describing how it is measured, we first show how variance affects the expected change in loss.

The relevance of estimator variance can be seen from a second-order approximation of the loss change. Let $L(\theta)$ be twice differentiable and consider the update

$$\theta^+ = \theta + \Delta\theta, \quad \Delta\theta = -\eta\hat{g},$$

where $\eta > 0$ is the learning rate. A second-order Taylor expansion around θ gives

$$L(\theta^+) \approx L(\theta) + \nabla_{\theta} L(\theta)^{\top} \Delta\theta + \frac{1}{2} \Delta\theta^{\top} H \Delta\theta,$$

where $H = \nabla_{\theta}^2 L(\theta)$ is the Hessian. Writing $g = \nabla_{\theta} L(\theta)$ and substituting $\Delta\theta = -\eta\hat{g}$, we obtain

$$\Delta L = L(\theta^+) - L(\theta) \approx -\eta g^{\top} \hat{g} + \frac{\eta^2}{2} \hat{g}^{\top} H \hat{g}.$$

Taking expectation over the stochastic gradient estimator, and assuming $\mathbb{E}[\hat{g}] = g$, gives

$$\mathbb{E}[\Delta L] \approx -\eta \|g\|^2 + \frac{\eta^2}{2} g^\top H g + \frac{\eta^2}{2} \text{tr}(H\Sigma).$$

The important point is that estimator variance appears directly in the expected loss change through the term $\text{tr}(H\Sigma)$, where H is the Hessian of the loss with respect to the parameters and Σ is the covariance of the gradient estimator. This means that update noise is not merely random fluctuation around an otherwise good direction; it can directly reduce the expected progress made by an update. In particular, variance is most harmful when it lies in directions where the loss has high curvature, because those directions are weighted more strongly by H . Ideally, we would measure the full curvature-weighted quantity $\text{tr}(H\Sigma)$, but estimating the Hessian and the full covariance matrix is not practical for the repeated perturbation experiments used here. We therefore measure estimator variance itself as a tractable and directly relevant diagnostic of update noise.

Estimator variance is naturally defined through the covariance matrix of the vector-valued gradient estimator. For a gradient estimator $\hat{g}$, this covariance matrix is

$$\Sigma = \mathbb{E}[(\hat{g} - \mathbb{E}[\hat{g}])(\hat{g} - \mathbb{E}[\hat{g}])^\top].$$

When the estimator is unbiased for the gradient $g = \nabla_\theta L(\theta)$, this can be written as

$$\Sigma = \mathbb{E}[(\hat{g} - g)(\hat{g} - g)^\top].$$

This is useful here because the exact gradient can be computed using backpropagation. Instead of estimating variance by subtracting the empirical mean of the perturbation samples, we subtract the exact backpropagation direction. However, Σ is a full covariance matrix over all parameters, which is too large to report directly. We therefore reduce it to a scalar by averaging its diagonal entries. Each diagonal entry is the variance of one parameter update component, corresponding to one weight or bias. The resulting quantity,

$$\frac{1}{P} \text{tr}(\Sigma),$$

is the average variance per parameter.

The measurement procedure follows the same frozen-checkpoint structure as the cosine-similarity calculation, using the diagnostic checkpoints and task-specific values of K listed in Table 4. For each checkpoint, mini-batch, and perturbation method, the exact backpropagation descent direction u_{BP} is computed and flattened over all weights and biases. We then draw K independent perturbation-based gradient estimates,

$$\hat{u}_m^{(1)}, \dots, \hat{u}_m^{(K)},$$

using the same frozen parameters and the same mini-batch. The empirical estimate for one method, checkpoint, and mini-batch is then

$$\hat{V}_m = \frac{1}{K} \sum_{k=1}^K \frac{1}{P} \left\| \hat{u}_m^{(k)} - u_{\text{BP}} \right\|_2^2.$$

Here, the factor $1/P$ averages over parameters, while the factor $1/K$ averages over independent gradient estimates.

As for cosine similarity, this variance estimate is averaged over all mini-batches in one full pass through the training set and over random seeds, with the number of seeds given in Table 4. In the main results, the three checkpoint values are also averaged to give one final variance value per method and dataset. The summary tables report the mean $\pm$ standard deviation of this aggregate variance across seeds. Checkpoint-specific variance plots are also generated and included in the appendix, while the main results focus on the aggregate variance plots.

4.5 Hyperparameter selection and experimental protocol

Hyperparameter tuning is important because each method must be given a fair chance to learn. If tuning is not done carefully, the results may reflect the chosen hyperparameters more than the

learning rule itself. If the learning rate or perturbation scale is too small, learning can become unnecessarily slow or the loss differences can become too weak to give useful updates. If they are too large, the updates can become unstable or no longer reflect a useful local learning signal. Since the learning rate and perturbation scale interact in ways that are not always obvious or trivial, fair tuning requires a systematic search.

For each learning problem, the dataset split, preprocessing, network architecture, mini-batch size, and evaluation procedure are held fixed across learning rules. The tuned hyperparameters differ between methods:

- **Backpropagation:** learning rate η .
- **Weight perturbation:** learning rate η and perturbation scale σ .
- **Vanilla node perturbation:** learning rate η and perturbation scale σ .
- **Fan-in-scaled node perturbation:** learning rate η and perturbation scale σ .
- **Input-scaled node perturbation:** learning rate η and perturbation scale σ .

Each method is tuned separately for each task.

The tuning procedure was carried out in two phases. The first phase focused on selecting reasonable perturbation scales. This was done before tuning the learning rate, because the learning rate and perturbation scale interact in ways that can make training curves difficult to interpret. For example, slow learning could be caused by a learning rate that is too small, a perturbation scale that is too small, a perturbation scale that is too large, or some combination of these. To separate these effects as much as possible, we first evaluated candidate σ values using the frozen-estimator diagnostics from Section 4.4. These diagnostics measure cosine similarity and estimator variance without applying parameter updates, and therefore do not depend on the learning rate. For each task and perturbation method, the search began with a broad set of five candidate σ values. The candidates were evaluated across the three diagnostic checkpoints and averaged. If the results showed a clear trend, for example that larger σ values improved cosine similarity and reduced variance, the range was shifted accordingly. This process was repeated with progressively narrower ranges until three nearby σ values were selected for each method: a central value with the best overall diagnostic trade-off, and one slightly smaller and one slightly larger value. These three values were then used as the perturbation-scale axis in the training grid search.

The second phase combined the selected perturbation scales with learning-rate tuning. For each perturbation method, we chose three initial learning rates based on preliminary runs and practical experience with the learning algorithms, giving a 3×3 grid over learning rate and perturbation scale. The first goal in this phase was to check whether the learning-rate range was sensible. If the training curves generally improved when the learning rate increased, the range was shifted upward; if larger learning rates caused instability or worse final performance, the range was shifted downward. This stage was evaluated using actual training and test curves, with attention to convergence speed, final loss, and stability. Once the learning-rate range appeared reasonable, the full 3×3 grid was inspected and refined. The grid was narrowed until the results were reasonably consistent and did not show a clear monotonic trend toward one edge of the grid, such as consistently better performance at the highest learning rate or largest perturbation scale. The aim was to choose a final setting from the interior of a stable region, rather than from a boundary where further improvement might simply require extending the search range. Table 5 shows the final local grids and the selected hyperparameter settings used for the thesis results.

Table 5: Local hyperparameter grids used during tuning. For perturbation methods, the grid is the Cartesian product of the listed learning rates and perturbation scales. The selected setting is the final configuration used for the thesis results.

Task	Method	Learning-rate grid	σ grid	Selected setting
Sinus	BP	{0.075, 0.100, 0.125}	—	$\eta = 0.100$
Sinus	IS-NP	{0.125, 0.150, 0.175}	{0.125, 0.150, 0.175}	$\eta = 0.150, \sigma = 0.150$
Sinus	Fan-in NP	{0.100, 0.1125, 0.125}	{0.075, 0.100, 0.125}	$\eta = 0.1125, \sigma = 0.100$
Sinus	Vanilla NP	{0.0750, 0.0825, 0.0900}	{0.250, 0.300, 0.350}	$\eta = 0.0825, \sigma = 0.300$
Sinus	WP	{0.0550, 0.0675, 0.0800}	{0.200, 0.225, 0.250}	$\eta = 0.0675, \sigma = 0.225$
California housing	BP	{0.075, 0.100, 0.150}	—	$\eta = 0.100$
California housing	IS-NP	{0.0200, 0.0270, 0.0350}	{0.080, 0.100, 0.120}	$\eta = 0.0270, \sigma = 0.100$
California housing	Fan-in NP	{0.0150, 0.0175, 0.0200}	{0.045, 0.0525, 0.060}	$\eta = 0.0175, \sigma = 0.0525$
California housing	Vanilla NP	{0.0150, 0.0175, 0.0200}	{0.400, 0.450, 0.500}	$\eta = 0.0175, \sigma = 0.450$
California housing	WP	{0.0110, 0.0135, 0.0160}	{0.080, 0.100, 0.120}	$\eta = 0.0135, \sigma = 0.100$
MNIST	BP	{0.400, 0.500, 0.600}	—	$\eta = 0.500$
MNIST	IS-NP	{0.0450, 0.0510, 0.0570}	{0.030, 0.0325, 0.035}	$\eta = 0.0510, \sigma = 0.0325$
MNIST	Fan-in NP	{0.0100, 0.0122, 0.0140}	{0.0080, 0.00875, 0.0095}	$\eta = 0.0122, \sigma = 0.00875$
MNIST	Vanilla NP	{0.00450, 0.00563, 0.00675}	{0.110, 0.120, 0.130}	$\eta = 0.00563, \sigma = 0.120$
MNIST	WP	{0.00300, 0.00375, 0.00450}	{0.025, 0.0275, 0.030}	$\eta = 0.00375, \sigma = 0.0275$
CIFAR-10	BP	{0.075, 0.100, 0.125}	—	$\eta = 0.100$
CIFAR-10	IS-NP	{0.0100, 0.0120, 0.0140}	{0.016, 0.0175, 0.01875}	$\eta = 0.0120, \sigma = 0.0175$
CIFAR-10	Fan-in NP	{0.00550, 0.00667, 0.00800}	{0.0050, 0.0055, 0.0060}	$\eta = 0.00667, \sigma = 0.0055$
CIFAR-10	Vanilla NP	{0.00320, 0.00400, 0.00480}	{0.165, 0.190, 0.215}	$\eta = 0.00400, \sigma = 0.190$
CIFAR-10	WP	{0.00120, 0.00160, 0.00200}	{0.0110, 0.0125, 0.0140}	$\eta = 0.00160, \sigma = 0.0125$

4.6 Declaration of AI

In the work on this thesis, I have used ChatGPT as a text tool and Codex as a programming assistant. ChatGPT has been used for idea development, text structuring, suggestions for wording, language improvements, and support in discussing academic explanations and argumentation. Codex has been used for programming assistance, including help with code and debugging. All content and code generated with such support have been reviewed, edited, quality-assured, and academically assessed by me. ChatGPT and Codex have not been used to generate the thesis in its entirety, and I take full responsibility for the academic content, code, methods, and results presented in the thesis.

5 Results

This chapter presents the empirical results from the evaluation strategy described in Section 4. The purpose of the chapter is to report the observed training performance and estimator diagnostics for the methods tested in this thesis. The interpretation of these results, including how they relate to the research questions and theoretical contribution, is left to Chapter 6.

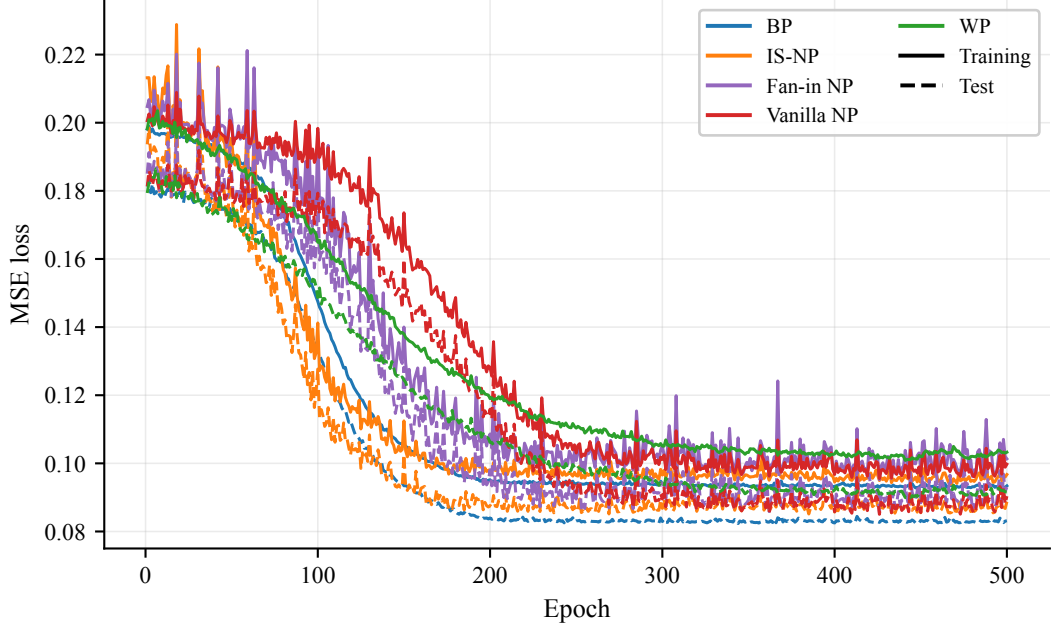
5.1 Sinus Regression

Figure 1 and Table 6 show that all methods were able to learn the sinus regression task well. The node perturbation methods converged to training and test losses close to the backpropagation baseline, with IS-NP and fan-in NP reaching nearly identical loss values. WP ended at a slightly higher loss level, but remained close to the other perturbation methods. IS-NP reached the convergence criterion at approximately the same epoch as backpropagation, while fan-in NP, vanilla NP, and WP converged somewhat later. The standard deviations reported for the best loss and score values are small, indicating little variation across seeds. The fan-in NP curves show somewhat larger fluctuations during training, but without signs of sustained instability. Overall, all perturbation methods worked well as learning algorithms on this task.

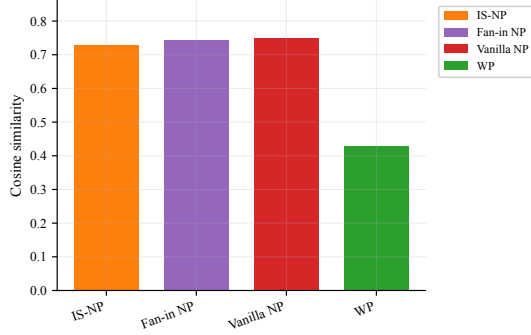
The estimator diagnostics show a clearer separation between WP and the node perturbation methods. The three node perturbation methods obtained similar cosine similarity and variance values, with only small differences between them. Vanilla NP had the highest cosine similarity, followed by fan-in NP and IS-NP, while the variance followed the same ordering, meaning that the methods with slightly higher cosine similarity also had slightly higher estimator variance. The differences between the node perturbation methods are therefore small and do not give a clear distinction between them. WP had both lower cosine similarity and higher estimator variance than all node perturbation methods. The reported standard deviations are small for both metrics, indicating that these diagnostic results are consistent across seeds.

Table 6: Main summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.

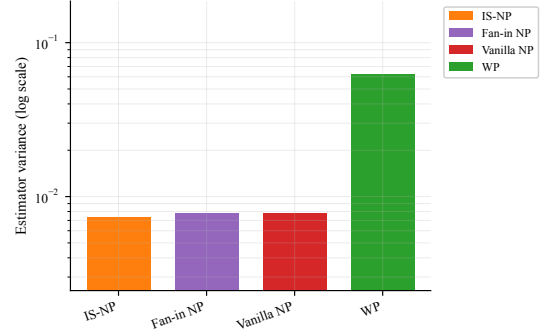
Method	Best test loss	Best test score	Cosine	Variance
BP	$0.0822 \pm 2.24\text{e-}4$	0.536 ± 0.00126	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.0825 \pm 8.83\text{e-}4$	0.534 ± 0.00498	0.728 ± 0.0685	$0.00735 \pm 5.18\text{e-}4$
Fan-in NP	$0.0825 \pm 8.34\text{e-}4$	0.534 ± 0.00471	0.744 ± 0.0634	$0.00787 \pm 4.40\text{e-}4$
Vanilla NP	$0.0830 \pm 8.56\text{e-}4$	0.532 ± 0.00483	0.749 ± 0.0656	$0.00784 \pm 4.38\text{e-}4$
WP	0.0865 ± 0.00357	0.512 ± 0.0202	0.429 ± 0.0913	0.0621 ± 0.00373



(a) Training and test loss.



(b) Cosine similarity.



(c) Estimator variance.

Figure 1: Results for the sinus regression task.

5.2 California Housing Regression

Figure 2 and Table 7 show that all methods were able to learn meaningful predictors on the California housing task, although the perturbation methods converged to higher training and test loss levels than backpropagation. Among the perturbation methods, the three node perturbation methods obtained very similar training and test performance, with no clear separation between them. WP reached the convergence criterion earliest among the perturbation methods, but ended with the highest training and test loss. The standard deviations reported for the best loss and score values are small for BP, IS-NP, vanilla NP, and WP, while fan-in NP shows somewhat larger variation across seeds.

The estimator diagnostics show a clear separation between WP and the node perturbation methods. Vanilla NP obtained the highest cosine similarity, followed closely by IS-NP, while fan-in NP was lower than both. WP had a substantially lower cosine similarity than all node perturbation methods. The variance ranking follows a similar pattern: IS-NP and vanilla NP obtained the lowest estimator variance values, fan-in NP was higher, and WP had the highest variance. The clearest distinction is therefore between WP and the node perturbation methods, but fan-in NP is also separated from IS-NP and vanilla NP on both diagnostic metrics.

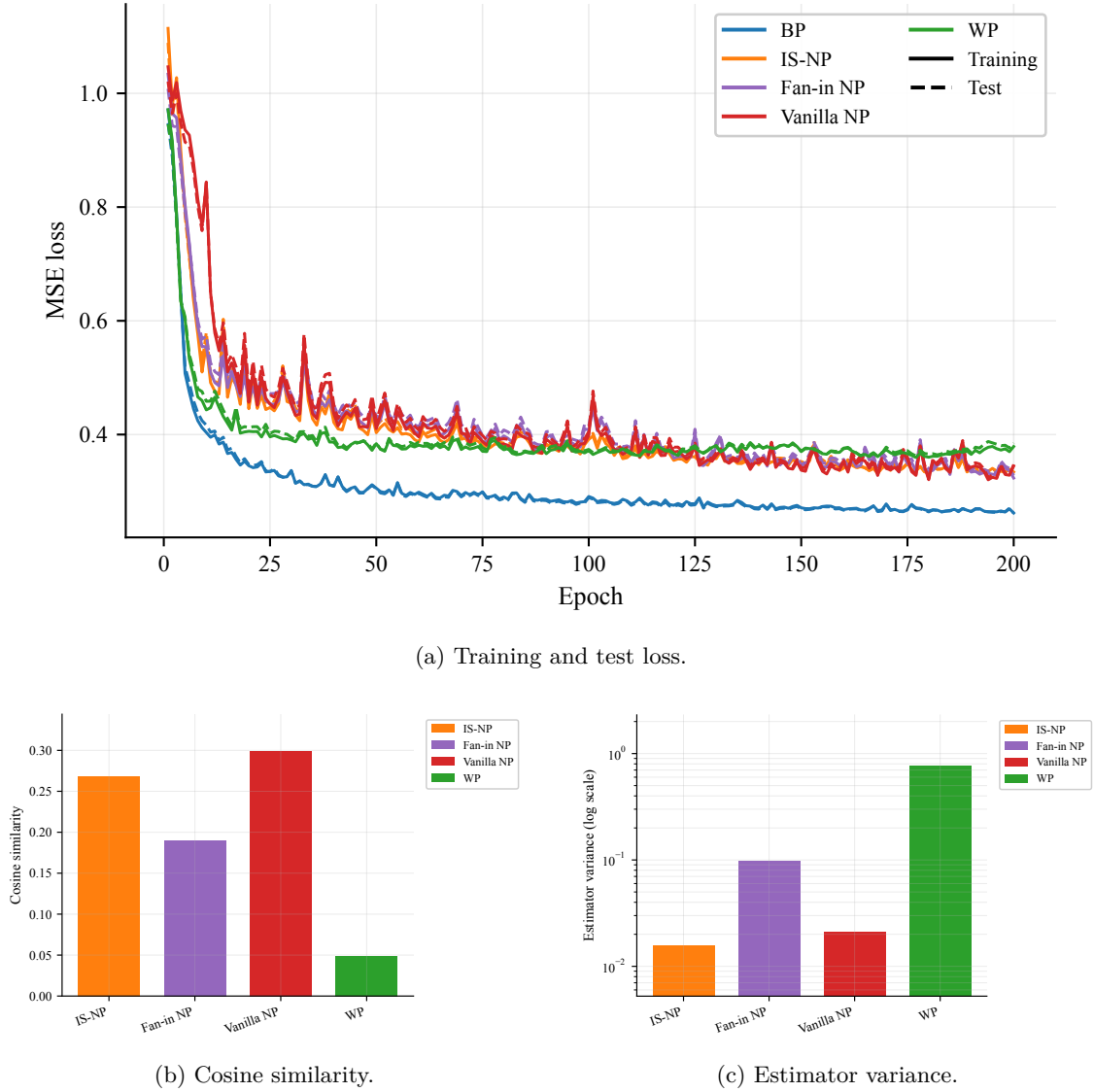


Figure 2: Results for the California housing regression task.

Table 7: Main summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.

Method	Best test loss	Best test score	Cosine	Variance
BP	0.261 ± 0.00253	0.733 ± 0.00259	1.00 ± 0.00	0.00 ± 0.00
IS-NP	0.324 ± 0.00597	0.668 ± 0.00613	0.268 ± 0.0346	$0.0157 \pm 3.71\text{e-}4$
Fan-in NP	0.320 ± 0.0139	0.671 ± 0.0143	0.190 ± 0.0234	0.0969 ± 0.0150
Vanilla NP	0.313 ± 0.00683	0.678 ± 0.00701	0.299 ± 0.0214	0.0212 ± 0.00216
WP	0.342 ± 0.00630	0.649 ± 0.00647	0.0490 ± 0.00478	0.769 ± 0.0182

5.3 MNIST Classification

Figure 3 and Table 8 show that MNIST is the first task where the differences between the learning rules become clearly visible in both training and test performance. All methods learn meaningful representations, with test accuracies well above the 10% chance level, but they do not learn equally well or equally fast. The ordering is clear across both loss and accuracy: backpropagation performs best, followed by IS-NP, fan-in NP, vanilla NP, and WP. Backpropagation reaches the highest test accuracy at 98.5%, while IS-NP obtains the best perturbation-based performance with a test accuracy of 96.2%. Fan-in NP, vanilla NP, and WP reach progressively lower accuracy and higher test loss. The WP loss curve also begins to increase toward the end of training, suggesting that the noisier update estimates are close to becoming unstable in this setting.

The estimator diagnostics follow the same overall ordering as the performance metrics. IS-NP obtains the highest cosine similarity and the lowest estimator variance among the perturbation methods, followed by fan-in NP, vanilla NP, and WP. The difference between IS-NP and fan-in NP is not large, especially for cosine similarity, but IS-NP is still best on both diagnostic metrics. Vanilla NP is clearly weaker than the two scaled node perturbation methods, with lower cosine similarity and higher variance. WP is separated most strongly from the other perturbation methods, with substantially lower cosine similarity and much higher estimator variance.

Table 8: Main summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.

Method	Best test loss	Best test score	Cosine	Variance
BP	$0.00397 \pm 1.76\text{e-}5$	$0.985 \pm 3.78\text{e-}4$	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.00943 \pm 2.89\text{e-}4$	$0.962 \pm 9.50\text{e-}4$	0.207 ± 0.0151	$1.28\text{e-}5 \pm 3.64\text{e-}7$
Fan-in NP	$0.0123 \pm 5.32\text{e-}4$	0.948 ± 0.00229	0.202 ± 0.0116	$1.54\text{e-}5 \pm 3.44\text{e-}7$
Vanilla NP	$0.0171 \pm 4.56\text{e-}4$	0.929 ± 0.00219	0.149 ± 0.00333	$3.93\text{e-}5 \pm 4.01\text{e-}7$
WP	0.0377 ± 0.00163	0.821 ± 0.00404	$0.00965 \pm 4.47\text{e-}4$	$0.00821 \pm 1.62\text{e-}4$

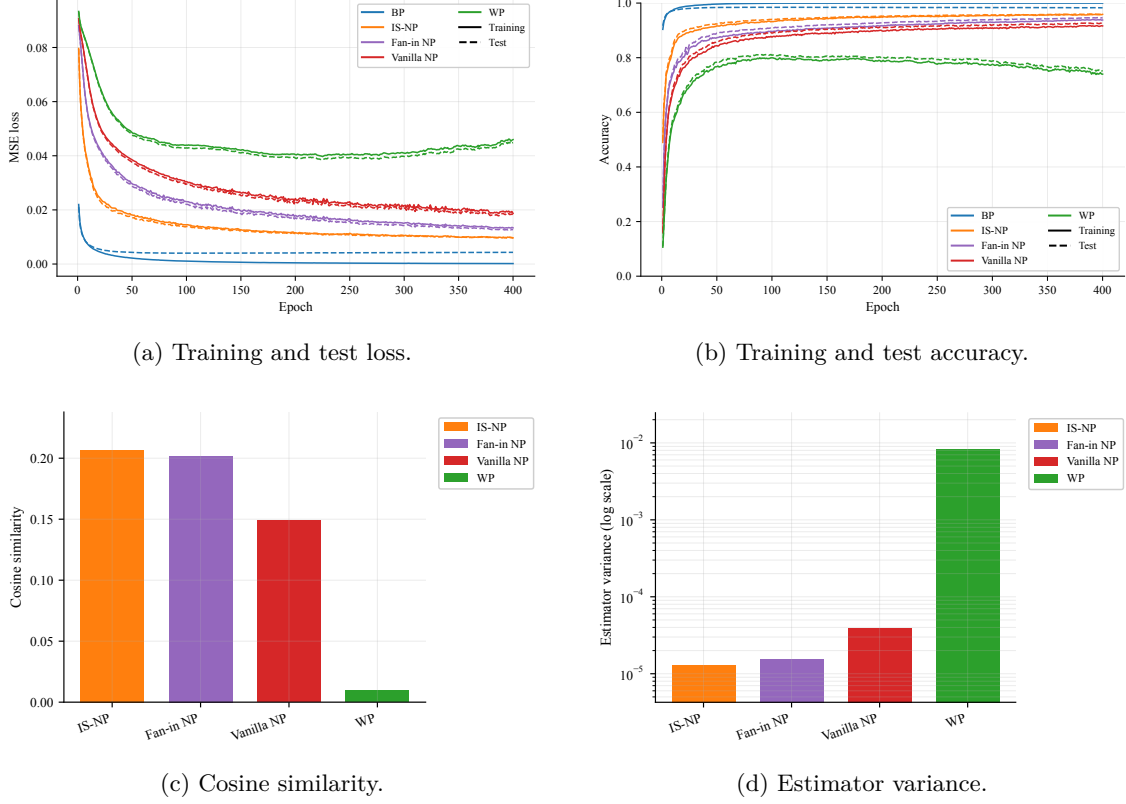
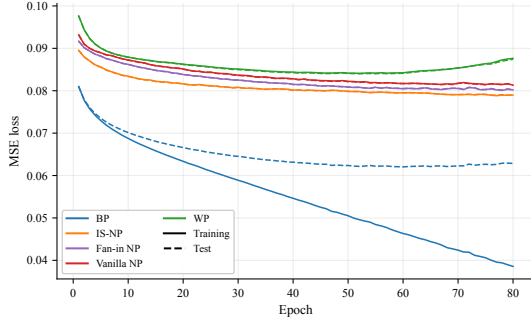


Figure 3: Results for the MNIST classification task.

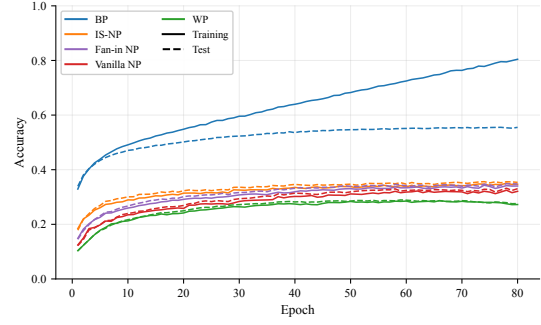
5.4 CIFAR-10 Classification

Figure 4 and Table 9 show that the differences between methods remain clear on the most difficult task. The gap between backpropagation and the perturbation methods is larger on CIFAR-10 than on MNIST. The perturbation methods still learn useful structure and obtain test accuracies well above the 10% chance level, with IS-NP reaching the strongest perturbation-based performance at 36.2%. However, all perturbation methods remain far behind backpropagation, which reaches a test accuracy of 55.8%. Although the four perturbation-method curves appear close in absolute terms, the total loss decrease on CIFAR-10 is modest, so these differences are still meaningful and consistent. The perturbation methods follow the same ordering as on MNIST: IS-NP performs best, followed by fan-in NP, vanilla NP, and WP.

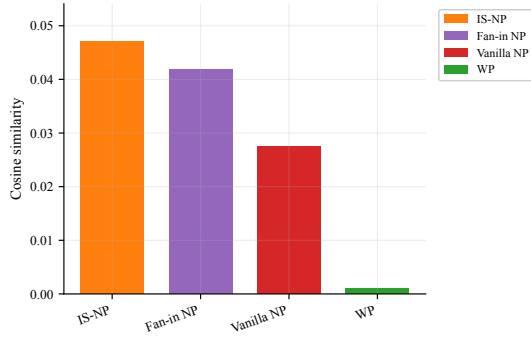
The estimator diagnostics follow the same ordering as the performance metrics. IS-NP obtains the highest cosine similarity and the lowest estimator variance among the perturbation methods, followed by fan-in NP, vanilla NP, and WP. As on MNIST, the difference between IS-NP and fan-in NP is relatively small, especially compared with the gap to vanilla NP and WP, but IS-NP remains best on both diagnostic metrics. Vanilla NP is weaker than the two scaled node perturbation methods, with lower cosine similarity and higher variance. WP is again separated most strongly from the other perturbation methods, with substantially lower cosine similarity and much higher estimator variance.



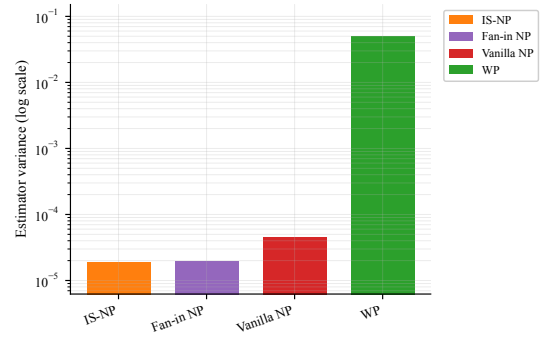
(a) Training and test loss.



(b) Training and test accuracy.



(c) Cosine similarity.



(d) Estimator variance.

Figure 4: Results for the CIFAR-10 classification task.

Table 9: Main summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.

Method	Best test loss	Best test score	Cosine	Variance
BP	$0.0618 \pm 7.69\text{e-}5$	0.558 ± 0.00303	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.0788 \pm 1.28\text{e-}4$	0.362 ± 0.00265	0.0470 ± 0.00208	$1.87\text{e-}5 \pm 4.35\text{e-}7$
Fan-in NP	$0.0799 \pm 4.49\text{e-}5$	0.354 ± 0.00195	0.0419 ± 0.00297	$1.97\text{e-}5 \pm 5.35\text{e-}7$
Vanilla NP	$0.0812 \pm 1.47\text{e-}4$	0.337 ± 0.00315	$0.0275 \pm 7.16\text{e-}4$	$4.44\text{e-}5 \pm 1.00\text{e-}6$
WP	$0.0838 \pm 3.85\text{e-}4$	0.296 ± 0.00637	$9.92\text{e-}4 \pm 4.38\text{e-}5$	$0.0504 \pm 6.81\text{e-}4$

5.5 Scaled Input Sensitivity

Figure 5 shows the effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$. For IS-NP and WP, the input scaling has little effect on either cosine similarity or estimator variance. In contrast, vanilla NP and fan-in NP both show substantially lower cosine similarity and higher estimator variance after the input range is scaled.

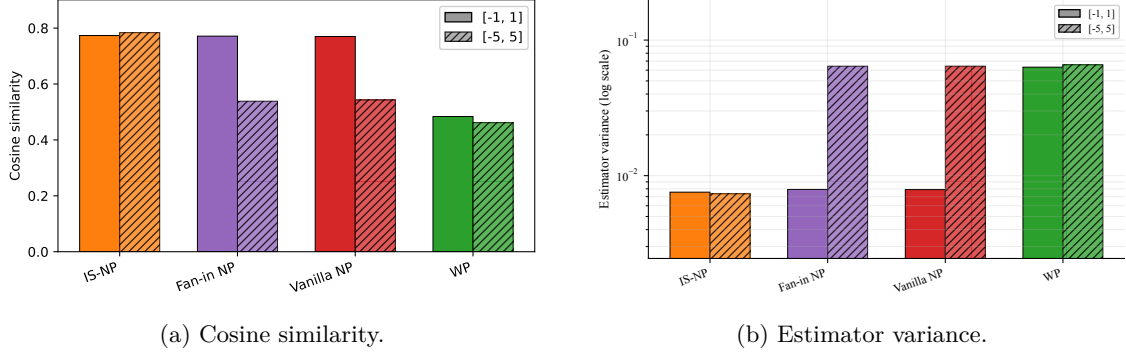


Figure 5: Effect of scaling the sinus input range from $[-1, 1]$ to $[-5, 5]$.

6 Discussion

This chapter discusses the empirical results presented above. The discussion is organized around the three questions that motivated the evaluation in Chapter 4: how useful perturbation-based methods are compared to backpropagation, how WP compares with NP in practice, and whether IS-NP improves over existing NP variants. We first give a brief overview of the main findings for each of these questions. The following subsections then discuss each finding in more detail, before turning to the limitations of the thesis and directions for future work.

6.1 Overview of the main findings

The first main finding is that perturbation-based learning methods are capable of training non-trivial multilayer networks, but remain clearly less effective than backpropagation as task difficulty increases. On the sinus regression task, the perturbation methods follow the backpropagation baseline closely, and on California housing and MNIST they still learn meaningful predictors or representations. However, they generally converge more slowly and to worse final loss and score values than backpropagation. On CIFAR-10, the gap becomes much larger: the perturbation methods learn above chance level, but do not approach the performance of backpropagation. This directly addresses Objective 3 by showing both where the perturbation methods work in the present experiments and where they begin to break down.

The second main finding is that the results consistently favour node perturbation over weight perturbation. Across the experiments, WP has higher estimator variance, lower cosine similarity, and weaker learning performance than the node perturbation methods. This difference is visible on the simpler tasks and becomes clearer on the more difficult classification tasks. These results are consistent with the variance-reduction argument developed in Chapter 3, where node-level perturbations are shown to reduce estimator variance relative to weight-level perturbations under the matched perturbation construction. The experiments therefore provide empirical support for the theoretical comparison between WP and NP, which is part of Objective 2.

The third main finding is that IS-NP is the strongest perturbation method overall. IS-NP obtains the highest cosine similarity, the lowest estimator variance across all tasks, and the strongest results on the more difficult classification tasks. The scaled-input diagnostic also separates IS-NP from vanilla NP and fan-in NP: when the input range is scaled, IS-NP remains stable while the

diagnostics for the other NP variants deteriorate. This supports the motivation for input-scaled perturbations developed in Chapter 3, where IS-NP was introduced to match the pre-activation noise induced by WP. The empirical results therefore support the second part of Objective 2: developing and evaluating a theoretically motivated improvement to existing node perturbation methods.

6.2 Practical capability and limits of perturbation-based learning

Taken together, the results show that perturbation-based methods are capable of genuine learning in multilayer networks, but that their practical range is limited compared with backpropagation. The task progression is informative because it shows where this limitation appears. On sinus regression, the perturbation methods remain close to BP, showing that scalar-feedback updates can work well when the learning problem is simple. On California housing and MNIST, the methods still learn meaningful predictors or representations, but the gap to BP becomes clearer. On CIFAR-10, the limitation becomes decisive: the methods learn above chance, but do not solve the task well in the MLP setup used here. The overall pattern is therefore not that perturbation methods fail outright, but that their usefulness decreases as the learning problem demands more accurate gradient information.

This trend is best explained by the quality of the gradient estimates. Backpropagation computes the exact gradient, while perturbation methods estimate it from random perturbations and a scalar loss signal. As the task and model become more complex, the useful update direction becomes harder to recover from such limited information. This is reflected in the cosine similarity diagnostics: the best perturbation estimates are roughly around 0.75 on sinus regression, around 0.3 on California housing, around 0.2 on MNIST, and close to 0.05 on CIFAR-10. Part of this decline is expected in higher-dimensional parameter spaces, where noisy estimates are more likely to be nearly orthogonal to the true gradient. However, this is precisely why directional alignment is informative: it measures whether the perturbation update contains enough signal to point in a useful descent direction. The raw variance values are less straightforward to compare across tasks, since they depend on factors such as loss scale, gradient magnitude, architecture, parameter dimension, and perturbation scale. Variance is therefore most useful for comparing methods within the same task, while cosine similarity gives the clearest indication of how informative the perturbation updates are across tasks.

The practical interpretation is that perturbation-based learning has a meaningful but limited operating range in the present setup. The fact that the methods can learn MNIST-level tasks suggests that scalar-feedback learning is not restricted to trivial problems, and may be useful in settings where backpropagation is unavailable or biologically implausible. At the same time, the current perturbation methods studied here, including IS-NP, are far from matching backpropagation on more complex tasks. If perturbation-based learning is to become part of a system that can compete with backpropagation-trained networks on such tasks, it will likely require substantial algorithmic improvements, or it must operate together with other forms of learning.

6.3 Weight versus node perturbation

The comparison between WP and NP gives one of the clearest links between the theoretical and empirical parts of the thesis. In Chapter 3, we showed that WP contains additional weight-level randomness compared with the matched node-level estimator IS-NP. The estimator diagnostics show this effect very clearly in practice. WP has substantially higher estimator variance than IS-NP across the experiments, often by orders of magnitude. The same pattern also appears for the other NP variants: even though the formal variance theorem was derived specifically for IS-NP, vanilla NP and fan-in-scaled NP also have much lower variance than WP in the empirical results. This suggests that the variance disadvantage of perturbing individual weights is not only a theoretical construction, but a strong practical effect in the networks studied here.

This variance difference also appears to matter for learning performance. On the simpler tasks,

WP still learns reasonably well, which shows that its higher variance is not always enough to prevent useful learning. In favourable settings, even noisy weight-level perturbations can provide a usable update direction. However, as the tasks become more difficult, the disadvantage becomes more visible. WP generally has weaker directional alignment and worse learning performance than the NP methods, especially on the image-classification tasks. This indicates that the additional variance is not just a diagnostic artifact: it affects the optimization dynamics when the learning problem requires more accurate and reliable update directions.

Taken together, these results substantially clarify the WP–NP relationship identified as incomplete in Chapter 2. The thesis does not only show empirically that NP performs better than WP in the tested tasks; it also provides a theoretical explanation for why this should happen. WP and IS-NP estimate the same perturbation-smoothed objective, but WP does so with higher coordinate-wise variance. The empirical results then show the same variance effect directly, and also show that it translates into weaker learning performance as task complexity increases. For the feedforward nonlinear multilayer networks studied here, this strongly supports NP as the preferable perturbation family when both WP and NP are viable. WP remains conceptually important, and may still be relevant in settings where parameter-level perturbations are more natural. However, when both WP and NP are viable, the results suggest that node-level perturbation is the more promising starting point for further development.

6.4 IS-NP versus other node perturbation methods

The comparison between IS-NP and the existing NP variants provides empirical support for the method developed in Chapter 3. As in the WP–NP comparison above, this separation is not equally visible on the simplest tasks, because all perturbation methods are still able to produce useful updates when the learning problem is favourable. The clearest separation appears on the more challenging tasks. There, IS-NP clearly performs better than fan-in-scaled NP, which in turn performs better than vanilla NP, both in the estimator diagnostics, the final test loss, and convergence speed. This is important because IS-NP was not introduced as an ad hoc modification, but derived from the theoretical relationship between WP and NP. The empirical results therefore suggest that the theoretical development in Chapter 3 leads to a practically useful perturbation method.

The most likely explanation is the input-scaling argument developed in Chapter 3. Vanilla NP uses the same perturbation scale regardless of the number or size of incoming activations, so its node noise is poorly matched to the weight-level perturbations it is meant to approximate. Fan-in-scaled NP improves on this by correcting for layer width, but it still assumes that the relevant input norm is well represented by its expected value. IS-NP goes one step further by scaling the perturbation using the actual incoming activation norm for each layer and input. This gives a natural explanation for the ordering observed on the harder tasks: vanilla NP ignores both layer width and activation scale, fan-in NP accounts for layer width but not the actual activation norm, and IS-NP accounts for both. The scaled-input diagnostic supports this interpretation in a controlled setting. When the sinus input range is changed from $[-1, 1]$ to $[-5, 5]$, the underlying regression problem is not fundamentally changed, but the activation scale is. As predicted, the cosine similarity of vanilla NP and fan-in NP deteriorates and their estimator variance increases, while IS-NP remains much more stable. This does not prove that input scaling is the only reason for the ordering in the main experiments, but it shows that when the assumptions behind the simpler NP variants are broken, their estimator diagnostics worsen in exactly the expected direction. Since activation norms in the main experiments vary across samples, layers, and training, these assumptions are unlikely to hold exactly there either. It is therefore reasonable to expect the same tendency: fan-in NP becomes less well matched than IS-NP, and vanilla NP becomes less well matched still, leading to worse directional alignment and higher estimator variance, which is exactly what we observe.

At the same time, this comparison should not be interpreted as a formal proof that IS-NP always dominates the other NP variants. Chapter 3 proves a variance result for IS-NP relative to WP, but the comparison with vanilla NP and fan-in-scaled NP is based on scaling arguments and empirical evidence. Other architectures, normalization schemes, perturbation scales, or tasks could change the relative performance of the NP methods. The appropriate conclusion is therefore that IS-

NP is the most promising node perturbation method tested in this thesis. The results strongly suggest that its input-dependent scaling is useful, and that the new method derived from the WP-NP analysis improves over the NP variants currently considered in the perturbation-learning literature. However, this does not prove that the same ordering will hold generally in all settings.

6.5 Limitations

One limitation is that all experiments were performed using MLPs and a limited set of supervised learning problems. This was useful for isolating the behaviour of the learning rules in a controlled setting, but it also limits how far the empirical conclusions can be generalized. Modern neural networks often rely on architectural features such as convolutions, attention, residual connections, or recurrent structure, and these may interact differently with perturbation-based learning. Similarly, the four tasks used in this thesis were chosen to span increasing difficulty, but they do not cover the full range of problems on which biologically plausible learning rules could be relevant.

A second limitation is that biological plausibility is evaluated at an abstract algorithmic level. The thesis studies whether learning rules based on stochastic perturbations, local information, and scalar feedback can train standard artificial neural networks. However, artificial neural networks remain highly simplified models of biological neural systems. Real neural circuits are recurrent, dynamic, spiking, chemically modulated, and embedded in bodies and environments. The results therefore support perturbation-based learning as a biologically motivated principle, but they do not show that the specific algorithms studied here are directly implementable in the brain.

A third limitation is that the experiments use fixed algorithmic choices. For practical reasons, the thesis had to choose specific baselines, batch sizes, initialization schemes and update rules. These choices were kept relatively simple in order to make the comparison between methods clear. However, perturbation-based learning may be sensitive to such design choices, and other variants could perform differently. The results should therefore be interpreted as an evaluation of a controlled set of perturbation methods, not as an exhaustive search over all possible implementations.

6.6 Future work

A natural direction for future work is to test perturbation-based learning on more architectures and more tasks. In particular, applying IS-NP and the other perturbation methods to CNNs, residual networks, recurrent networks, or transformer-like architectures would give a better understanding of whether the results found here extend beyond MLPs. Testing on a wider range of datasets would also help clarify where perturbation-based methods break down and which task properties make them more or less effective.

Another important direction is to improve the optimization behaviour of perturbation methods. Modern backpropagation-based training benefits greatly from techniques such as momentum, adaptive learning rates, learning-rate schedules, normalization, and architectural choices such as skip connections. Perturbation methods may benefit from analogous improvements, such as adaptive perturbation scales, decaying noise schedules, layerwise perturbations, or optimizer variants inspired by momentum or Adam. It would also be useful to test whether other ideas from the perturbation-learning articles presented in Table 1 can be combined with the scaling approach studied in this thesis, including noisy-baseline methods, weight normalization, and decorrelation of the inputs. At the same time, such modifications should be evaluated not only by their empirical performance, but also by whether they preserve the biological motivation of the learning rule.

A third direction is to test perturbation-based learning in more biologically realistic settings. Since artificial neural networks are only simplified abstractions of biological neural systems, it would be valuable to study whether similar learning principles can be implemented in recurrent, spiking, neuromorphic, or biological-substrate-inspired architectures. Systems such as CL1 illustrate that increasingly biological forms of computation are becoming experimentally relevant. Even if the exact algorithms studied in this thesis cannot be applied directly to such systems, testing related perturbation-and-feedback principles in more biological architectures would be highly informative.

Finally, future work should compare perturbation-based learning more broadly against other biologically plausible alternatives to backpropagation. This thesis focuses on WP, NP, fan-in-scaled NP, and IS-NP in depth, but methods such as feedback alignment, predictive coding, local losses, and the forward-forward algorithm represent other possible routes away from standard backpropagation. A systematic comparison across such methods would make it clearer how perturbation learning performs both as a practical training method and as a biologically plausible learning principle.

7 Conclusion

This thesis investigated perturbation-based learning as a biologically motivated alternative to backpropagation. The work was organized around three research questions, which are answered directly below.

What has been proposed and theoretically established in the literature on perturbation-based learning? The thesis concludes that perturbation-based learning is an established but still incomplete research direction. In Chapter 2, we identified 21 different update rules across 9 articles and organized them in a common notation. This showed that there exist many different versions of perturbation-based learning, with variants that differ in where noise is injected, how the learning signal is formed, and how the update is scaled. The two central families of perturbation methods are weight perturbation, where noise is applied directly to the parameters, and node perturbation, where noise is applied to neural activity or pre-activations. However, despite the existence of many relevant articles and update rules, the relationship between WP and NP remains insufficiently resolved in the current literature. While previous work provides an analytical comparison showing why NP can be expected to outperform WP in a simplified one-layer linear setting, no corresponding theoretical derivation appears to exist for how the two families compare in general nonlinear multilayer networks. The conclusion is therefore that perturbation-based learning is well established as a family of learning rules, but that its theoretical foundation remains incomplete because the relationship between its two central families has not been fully clarified.

How can perturbation-based learning be further developed? This is the most important question of the thesis. The answer is that perturbation-based learning can be further developed by clarifying the relationship between WP and NP, which was identified above as an incomplete part of the existing literature. Inspired by the local reparameterization trick of Kingma et al. (2015), Chapter 3 showed that the weight-level noise in WP can be translated into an equivalent pre-activation noise distribution. This led to the derivation of input-scaled node perturbation (IS-NP), a node-level update rule with the same expected update as WP, but with a variance that is theoretically guaranteed not to be larger and is expected to be lower in practice. The empirical results support this prediction: WP has much higher estimator variance than IS-NP, and performs clearly worse than the NP methods on the more difficult tasks. This suggests that the variance reduction is not only a theoretical result, but also matters for practical learning, and supports NP as the more promising perturbation-learning direction. IS-NP also performs best among the NP methods included in the evaluation, outperforming both vanilla NP and fan-in-scaled NP in the most challenging experiments. The conclusion is therefore that the thesis further develops perturbation-based learning in two connected ways: by strengthening its theoretical foundation through the WP–NP clarification, and by deriving a new perturbation method that appears practically promising in the settings studied here.

What is the practical capability of perturbation-based learning? The empirical results show that perturbation-based learning is capable of training nontrivial multilayer networks, including meaningful performance on MNIST classification using an MLP. However, the methods remain clearly limited compared with backpropagation. The gap is small on the simplest task, becomes more visible on California housing and MNIST, and is large on CIFAR-10, where the perturbation methods do not approach the speed or final performance of backpropagation. This suggests that current perturbation-based methods are not sufficient to train state-of-the-art models or solve highly complex supervised learning problems competitively on their own. Their signific-

ance is instead that they demonstrate that biologically motivated learning rules based on stochastic perturbations and scalar feedback can support genuine learning. This should not be interpreted as evidence that perturbation-based learning will necessarily become a central component of future AI systems. The experiments were carried out in standard artificial neural networks, not in biologically realistic architectures, and the broader relevance of these methods remains speculative. A more cautious conclusion is that perturbation-based learning may be one useful building block among several in future biologically inspired learning systems, much like three-factor learning is one learning mechanism among several in the brain.

Whether perturbation-based learning will ever become important for future AI systems is impossible to know. It may remain a biologically motivated curiosity, useful mainly for understanding a narrow class of learning rules. But it may also turn out that some of the next major steps in AI require ideas that look quite different from today's dominant recipe of larger models, larger datasets, and more backpropagation. If that future includes architectures inspired more deeply by biology, then perturbation-based learning could be one of the mechanisms that makes learning in such systems possible. The contributions of this thesis make that possibility a little more plausible.

8 Sustainability

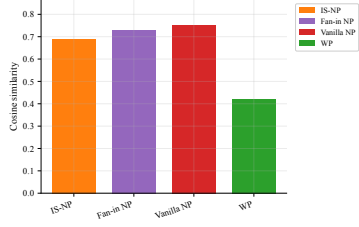
This thesis relates indirectly to sustainability through its motivation of more efficient and biologically inspired learning systems. Modern artificial intelligence systems require large amounts of computation and energy, and this challenge is likely to become more important as models continue to grow in size and capability. The methods studied in this thesis are not shown to be more computationally efficient than backpropagation in standard artificial neural networks, and the empirical results do not suggest that they are currently practical replacements. However, the broader research direction is still relevant to sustainability: biological learning demonstrates that powerful, adaptive intelligence can operate with far lower energy consumption than current digital systems. By studying learning rules that could be compatible with more biologically inspired architectures, this thesis contributes modestly to long-term research on alternative AI systems that may eventually be more energy-efficient than today's approaches.

Appendix

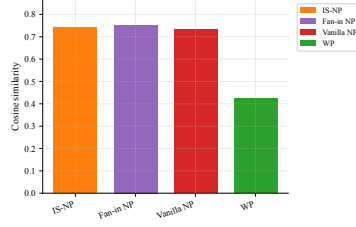
A Checkpoint-Specific Estimator Diagnostics

This appendix shows the checkpoint-specific cosine similarity and estimator variance plots used to support the aggregate diagnostics in Chapter 4 and Chapter 5. For each learning problem, the diagnostics are shown separately at the three frozen backpropagation checkpoints used in the empirical evaluation. The appendix also includes the checkpoint-specific plots for the scaled-input sinus diagnostic.

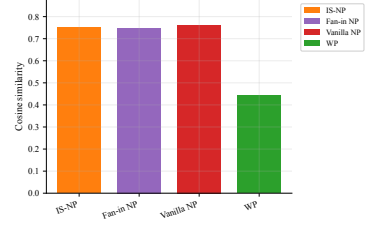
A.1 Sinus Regression



(a) Epoch 1.

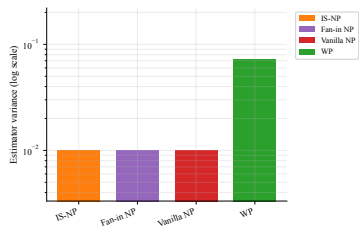


(b) Epoch 120.

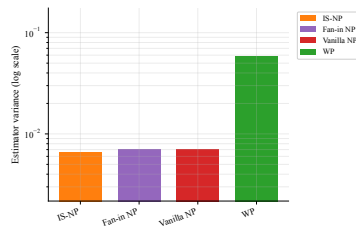


(c) Epoch 240.

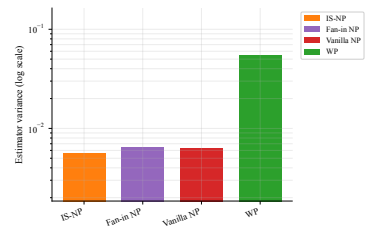
Figure 6: Checkpoint-specific cosine similarity for the sinus regression task.



(a) Epoch 1.



(b) Epoch 120.



(c) Epoch 240.

Figure 7: Checkpoint-specific estimator variance for the sinus regression task.

A.2 Scaled Input Sensitivity

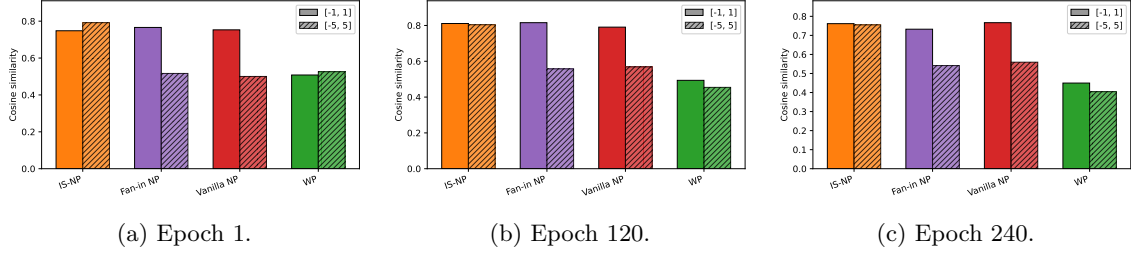


Figure 8: Checkpoint-specific cosine similarity for the scaled-input sinus diagnostic.

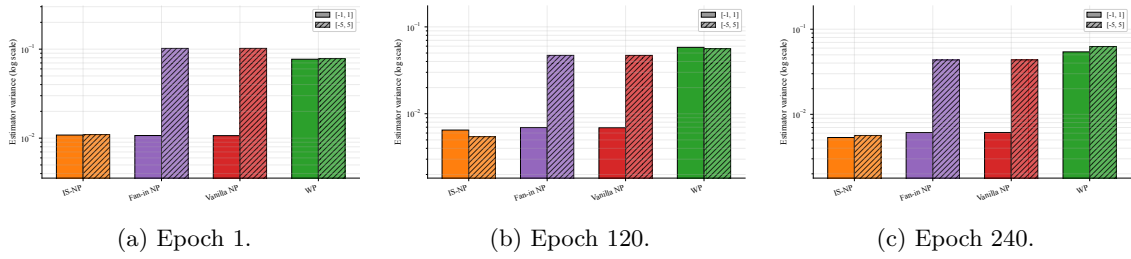


Figure 9: Checkpoint-specific estimator variance for the scaled-input sinus diagnostic.

A.3 California Housing Regression

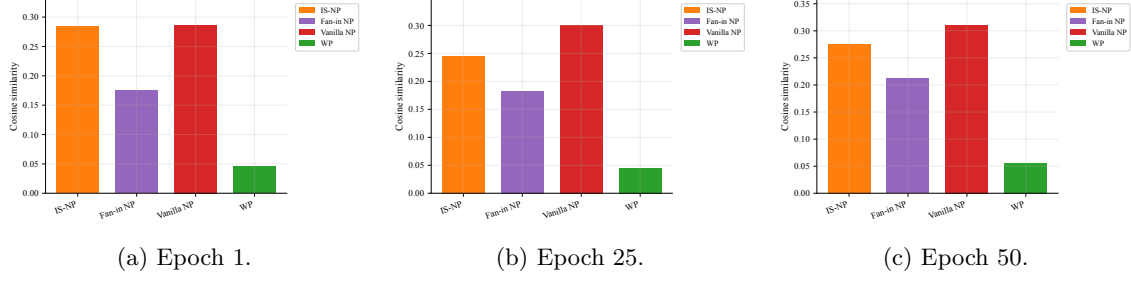


Figure 10: Checkpoint-specific cosine similarity for the California housing regression task.

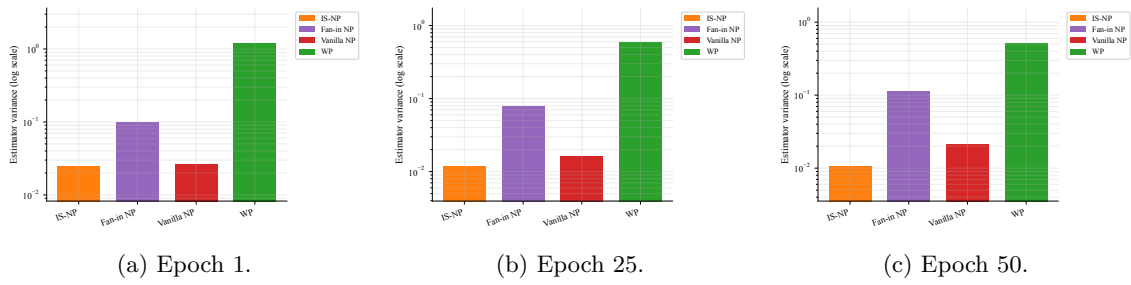
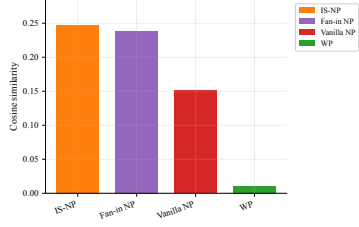
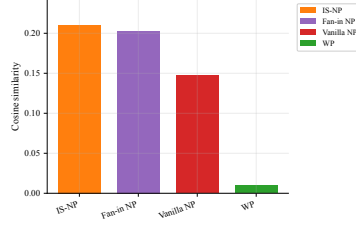


Figure 11: Checkpoint-specific estimator variance for the California housing regression task.

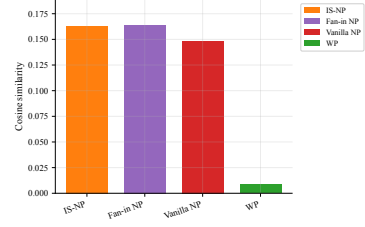
A.4 MNIST Classification



(a) Epoch 1.

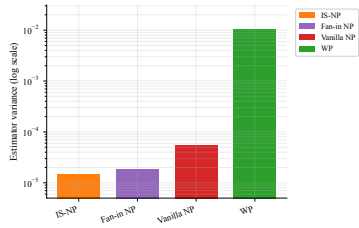


(b) Epoch 10.

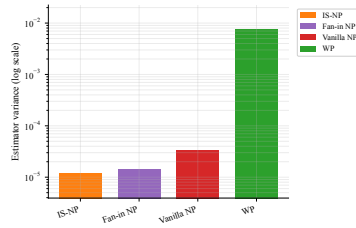


(c) Epoch 20.

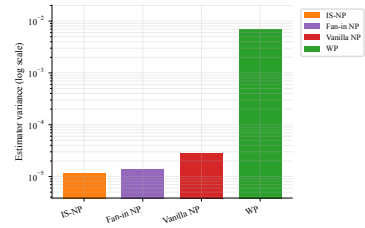
Figure 12: Checkpoint-specific cosine similarity for the MNIST classification task.



(a) Epoch 1.



(b) Epoch 10.



(c) Epoch 20.

Figure 13: Checkpoint-specific estimator variance for the MNIST classification task.

A.5 CIFAR-10 Classification

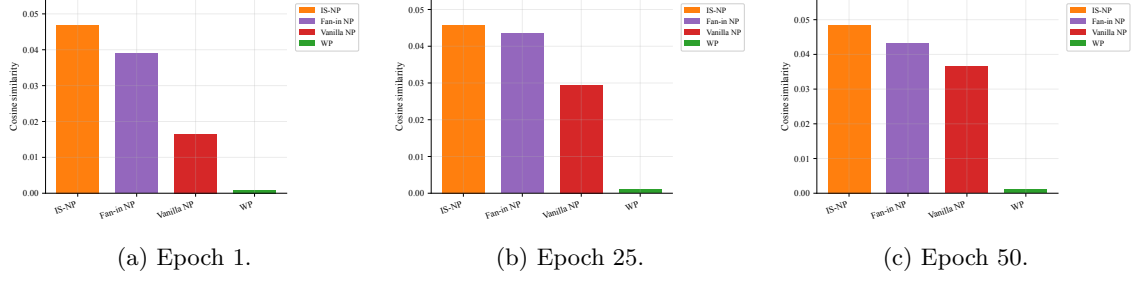


Figure 14: Checkpoint-specific cosine similarity for the CIFAR-10 classification task.

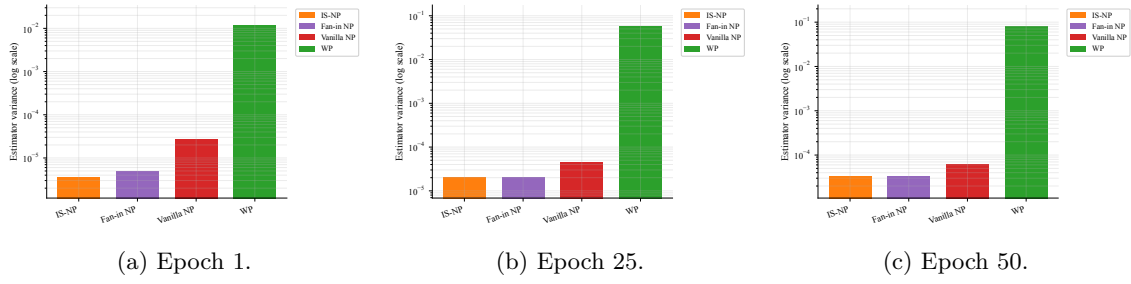


Figure 15: Checkpoint-specific estimator variance for the CIFAR-10 classification task.

B Full Result Summary Tables

This appendix contains the full numerical summary tables for the four learning problems. These tables include the metrics omitted from the compact tables in Chapter 5, namely the training metrics and convergence epoch.

Table 10: Full summary metrics for the sinus regression task. Values are mean $\pm$ standard deviation across seeds.

Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
BP	$0.0925 \pm 3.08\text{e-}4$	$0.0822 \pm 2.24\text{e-}4$	0.529 ± 0.00157	0.536 ± 0.00126	R2	133 ± 27	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.0921 \pm 6.77\text{e-}4$	$0.0825 \pm 8.83\text{e-}4$	0.531 ± 0.00345	0.534 ± 0.00498	R2	120 ± 26	0.728 ± 0.0685	$0.00735 \pm 5.18\text{e-}4$
Fan-in NP	$0.0931 \pm 5.70\text{e-}4$	$0.0825 \pm 8.34\text{e-}4$	0.526 ± 0.00290	0.534 ± 0.00471	R2	172 ± 29	0.744 ± 0.0634	$0.00787 \pm 4.40\text{e-}4$
Vanilla NP	$0.0936 \pm 9.59\text{e-}4$	$0.0830 \pm 8.56\text{e-}4$	0.523 ± 0.00489	0.532 ± 0.00483	R2	226 ± 34	0.749 ± 0.0656	$0.00784 \pm 4.38\text{e-}4$
WP	0.0975 ± 0.00324	0.0865 ± 0.00357	0.504 ± 0.0165	0.512 ± 0.0202	R2	229 ± 58	0.429 ± 0.0913	0.0621 ± 0.00373

Table 11: Full summary metrics for the California housing regression task. Values are mean $\pm$ standard deviation across seeds.

Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
BP	0.261 $\pm$ 0.00234	0.261 $\pm$ 0.00253	0.739 $\pm$ 0.00234	0.733 $\pm$ 0.00259	R2	26 $\pm$ 3	1.00 $\pm$ 0.00	0.00 $\pm$ 0.00
IS-NP	0.324 $\pm$ 0.00471	0.324 $\pm$ 0.00597	0.676 $\pm$ 0.00471	0.668 $\pm$ 0.00613	R2	59 $\pm$ 14	0.268 $\pm$ 0.0346	0.0157 $\pm$ 3.71e-4
Fan-in NP	0.316 $\pm$ 0.0134	0.320 $\pm$ 0.0139	0.684 $\pm$ 0.0134	0.671 $\pm$ 0.0143	R2	78 $\pm$ 17	0.190 $\pm$ 0.0234	0.0969 $\pm$ 0.0150
Vanilla NP	0.312 $\pm$ 0.00495	0.313 $\pm$ 0.00683	0.688 $\pm$ 0.00495	0.678 $\pm$ 0.00701	R2	82 $\pm$ 16	0.299 $\pm$ 0.0214	0.0212 $\pm$ 0.00216
WP	0.340 $\pm$ 0.00967	0.342 $\pm$ 0.00630	0.660 $\pm$ 0.00967	0.649 $\pm$ 0.00647	R2	20 $\pm$ 3	0.0490 $\pm$ 0.00478	0.769 $\pm$ 0.0182

Table 12: Full summary metrics for the MNIST classification task. Values are mean $\pm$ standard deviation across seeds.

Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
BP	$1.63\text{e-}4 \pm 1.96\text{e-}5$	$0.00397 \pm 1.76\text{e-}5$	1.00 ± 0.00	$0.985 \pm 3.78\text{e-}4$	Accuracy	19 ± 1	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.00954 \pm 3.17\text{e-}4$	$0.00943 \pm 2.89\text{e-}4$	0.960 ± 0.00265	$0.962 \pm 9.50\text{e-}4$	Accuracy	56 ± 4	0.207 ± 0.0151	$1.28\text{e-}5 \pm 3.64\text{e-}7$
Fan-in NP	$0.0130 \pm 5.69\text{e-}4$	$0.0123 \pm 5.32\text{e-}4$	0.941 ± 0.00318	0.948 ± 0.00229	Accuracy	122 ± 6	0.202 ± 0.0116	$1.54\text{e-}5 \pm 3.44\text{e-}7$
Vanilla NP	$0.0180 \pm 4.94\text{e-}4$	$0.0171 \pm 4.56\text{e-}4$	0.921 ± 0.00147	0.929 ± 0.00219	Accuracy	162 ± 17	0.149 ± 0.00333	$3.93\text{e-}5 \pm 4.01\text{e-}7$
WP	0.0388 ± 0.00145	0.0377 ± 0.00163	0.809 ± 0.00742	0.821 ± 0.00404	Accuracy	90 ± 30	$0.00965 \pm 4.47\text{e-}4$	$0.00821 \pm 1.62\text{e-}4$

Table 13: Full summary metrics for the CIFAR-10 classification task. Values are mean $\pm$ standard deviation across seeds.

Method	Best train loss	Best test loss	Best train score	Best test score	Score metric	Convergence epoch	Cosine	Variance
BP	$0.0385 \pm 4.14e-4$	$0.0618 \pm 7.69e-5$	0.805 ± 0.00513	0.558 ± 0.00303	Accuracy	35 ± 1	1.00 ± 0.00	0.00 ± 0.00
IS-NP	$0.0788 \pm 1.92e-4$	$0.0788 \pm 1.28e-4$	0.356 ± 0.00455	0.362 ± 0.00265	Accuracy	47 ± 3	0.0470 ± 0.00208	$1.87e-5 \pm 4.35e-7$
Fan-in NP	$0.0798 \pm 2.76e-5$	$0.0799 \pm 4.49e-5$	0.350 ± 0.00388	0.354 ± 0.00195	Accuracy	46 ± 3	0.0419 ± 0.00297	$1.97e-5 \pm 5.35e-7$
Vanilla NP	$0.0812 \pm 1.16e-4$	$0.0812 \pm 1.47e-4$	0.332 ± 0.00457	0.337 ± 0.00315	Accuracy	45 ± 2	$0.0275 \pm 7.16e-4$	$4.44e-5 \pm 1.00e-6$
WP	$0.0839 \pm 4.94e-4$	$0.0838 \pm 3.85e-4$	0.294 ± 0.0109	0.296 ± 0.00637	Accuracy	29 ± 6	$9.92e-4 \pm 4.38e-5$	$0.0504 \pm 6.81e-4$

C Expected Equivalence Between WP and IS-NP

This appendix derives the expected-equivalence result between WP and IS-NP. The derivation shows that the two methods induce the same distribution over perturbed forward passes, and therefore estimate the gradient of the same perturbation-smoothed objective. This result is not the central contribution of Chapter 3, but it strengthens the variance analysis in Section 3.3 by showing that WP and IS-NP can be compared as estimators with the same expected update.

Recall from Section 2.4 that WP does not optimize the deterministic loss directly, but the perturbation-smoothed objective

$$F(\theta) := \mathbb{E}_{\theta' \sim p_\theta} [L(\theta')], \quad (65)$$

where θ denotes the trainable network parameters, including both weights and biases. For WP, the perturbed parameters can be written as $\theta' = \theta + \Delta\theta$, so the objective is

$$F_{\text{WP}}(\theta) = \mathbb{E}_{\Delta\theta} [L(\text{forward}(\theta + \Delta\theta))]. \quad (66)$$

Although this expectation is defined over parameter perturbations, the loss does not depend on $\Delta\theta$ directly. It depends on $\Delta\theta$ only through the perturbed forward pass that these parameter perturbations produce.

Let

$$\Delta\mathbf{Z} = (\Delta\mathbf{z}^1, \dots, \Delta\mathbf{z}^L) \quad (67)$$

denote the full induced pre-activation noise field. Since $\Delta\mathbf{Z}$ determines the perturbed forward pass, WP can equivalently be viewed as inducing a distribution over $\Delta\mathbf{Z}$. If this induced distribution is denoted by $p_{\text{WP}}(\Delta\mathbf{Z})$, then the WP objective can be written as

$$F_{\text{WP}}(\theta) = \mathbb{E}_{\Delta\mathbf{Z} \sim p_{\text{WP}}} [L(\text{forward}(\theta; \Delta\mathbf{Z}))]. \quad (68)$$

Equivalently,

$$F_{\text{WP}}(\theta) = \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{WP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z}. \quad (69)$$

This expression makes clear that the objective is an average over possible perturbed forward passes: each possible pre-activation noise field contributes its loss value, weighted by the probability that WP induces that noise field.

From Section 3.2, WP induces the conditional distribution

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (70)$$

IS-NP is defined to sample directly from this same conditional distribution. Therefore, for each layer,

$$p_{\text{IS-NP}}\left(\Delta\mathbf{z}^\ell \mid \mathbf{x}^{\ell-1'}\right) = p_{\text{WP}}\left(\Delta\mathbf{z}^\ell \mid \mathbf{x}^{\ell-1'}\right). \quad (71)$$

The forward recursion is deterministic once $\Delta\mathbf{z}^\ell$ is sampled:

$$\mathbf{z}^{\ell'} = \mathbf{W}^\ell \mathbf{x}^{\ell-1'} + \mathbf{b}^\ell + \Delta\mathbf{z}^\ell, \quad \mathbf{x}^{\ell'} = \phi(\mathbf{z}^{\ell'}). \quad (72)$$

Hence the equality of the layer-wise conditional distributions propagates through the network, giving the same distribution over the full pre-activation noise field:

$$p_{\text{IS-NP}}(\Delta\mathbf{Z}) = p_{\text{WP}}(\Delta\mathbf{Z}). \quad (73)$$

The IS-NP objective can therefore be written as

$$F_{\text{IS-NP}}(\theta) = \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{IS-NP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z} \quad (74)$$

$$= \int L(\text{forward}(\theta; \Delta\mathbf{Z})) p_{\text{WP}}(\Delta\mathbf{Z}) d\Delta\mathbf{Z} \quad (75)$$

$$= F_{\text{WP}}(\theta). \quad (76)$$

Thus, WP and IS-NP define the same perturbation-smoothed objective. For every possible $\Delta \mathbf{Z}$, the two methods assign the same probability to the corresponding perturbed forward pass, and the loss value produced by that forward pass is the same.

Since both WP and IS-NP are unbiased score-function estimators of the gradient of this same objective, their expected gradient estimates are equal:

$$\mathbb{E} [\hat{g}^{\text{WP}}] = \nabla F(\theta) = \mathbb{E} [\hat{g}^{\text{IS-NP}}]. \quad (77)$$

In other words, WP and IS-NP have the same expected update direction. The difference between them is therefore not the objective they estimate, but the variance of their gradient estimators.

The same argument does not generally apply to standard NP or fan-in-scaled NP. Standard NP samples fixed-variance pre-activation noise,

$$\Delta z_j^\ell \sim \mathcal{N}(0, \sigma^2), \quad (78)$$

whereas WP induces

$$\Delta z_j^\ell \mid \mathbf{x}^{\ell-1'} \sim \mathcal{N}\left(0, \sigma^2 \left(1 + \|\mathbf{x}^{\ell-1'}\|^2\right)\right). \quad (79)$$

These distributions are equal only in the special case where

$$\|\mathbf{x}^{\ell-1'}\|^2 = 0. \quad (80)$$

Similarly, fan-in-scaled NP samples noise with variance proportional to the number of incoming units,

$$\Delta z_j^\ell \sim \mathcal{N}\left(0, \sigma^2(N_{\ell-1} + 1)\right). \quad (81)$$

This matches the WP-induced distribution only when

$$\|\mathbf{x}^{\ell-1'}\|^2 = N_{\ell-1}. \quad (82)$$

Thus, standard NP and fan-in-scaled NP generally induce different distributions over pre-activation perturbations than WP and IS-NP. They may match WP under special normalization assumptions, but not in the general case where the incoming activation norm varies across layers, samples, and training time.

This distinction provides useful context for the variance analysis in Section 3.3. Since WP and IS-NP estimate the gradient of the same perturbation-smoothed objective, comparing their variance is a clean estimator-level comparison: both estimators have the same expectation, and the only question is how much they vary around it. The same is not generally true for standard NP or fan-in-scaled NP, since these methods may target slightly different smoothed objectives. IS-NP therefore provides something that the existing NP variants do not: a node-level perturbation rule that can be compared directly with WP at the level of estimator variance.